

**Оптимизация запросов, часть 1.
Последовательное сканирование.
Индексы. Индексные структуры.
Алгоритмы доступа к данным. Битовые карты.**



Представление данных

Данные хранятся в файлах на жестких дисках.

Любой файл, используемый для объектов базы данных, делится на блоки одинаковой длины; по умолчанию PostgreSQL использует блоки по 8192 байта каждый (8 Кбайт).

Блок - это единица, которая передается между жестким диском и оперативной памятью, а количество операций ввода-вывода, необходимое для выполнения любого доступа к данным, равно количеству блоков, которые читаются или записываются.

Объекты базы данных состоят из логических элементов (строк таблиц, индексных записей и т. д.).

PostgreSQL выделяет место для этих элементов блоками. Несколько мелких элементов могут находиться в одном блоке; более крупные элементы могут быть распределены между несколькими блоками.



Представление данных

Общая структура блока показана на рисунке

Распределение элементов по блокам зависит от типа объекта базы данных.

Строки таблицы хранятся с использованием структуры данных, называемой кучей: строка может быть вставлена в любой блок, в котором достаточно свободного места, без специального упорядочивания.

Другие объекты (в частности, индексы) могут использовать блоки по-другому.

Заголовок блока

Указатель на логический элемент

Указатель на логический элемент

...

Свободное пространство

Логический элемент: строка таблицы

Логический элемент: строка таблицы

...

Логический элемент: индексная запись

Логический элемент: индексная запись

...

Конец блока



Последовательное (полное) сканирование таблиц

При полном сканировании движок базы данных последовательно считывает все строки в таблице и для каждой строки проверяет условие фильтрации. Стоимость этого алгоритма можно оценить на основе псевдокода:

```
FOR each block IN a_table
  read block
  FOR each row IN block
    IF filter_condition (row)
      THEN output (row)
    END IF
  END FOR
END FOR
```

Стоимостная модель физической операции оценивает ресурсы, необходимые для выполнения операции. Обычно стоимость зависит от таблиц, указанных в качестве аргументов операции. Будем использовать обозначения: для любой таблицы (отношения) R символы T и B обозначают количество строк в таблице и количество дисковых блоков, занимаемых таблицей, соответственно.

Количество операций ввода-вывода равно B ; общее количество итераций внутреннего цикла равно T . Нам также необходимо оценить стоимость операций, порождающих выходные строки. Она зависит от селективности (которая обозначается S) и равняется $S * T$. Стоимость полного сканирования:

$c1 * B + c2 * T + c3 * S * T$, где константы $c1$, $c2$ и $c3$ представляют характеристики аппаратного обеспечения.

Полностью просканировать можно любую таблицу; для этого не нужны дополнительные структуры данных. Остальные алгоритмы зависят от наличия индексов в таблице (см. далее).



План выполнения запроса, азы EXPLAIN

Выполняя любой полученный запрос, PostgreSQL разрабатывает для него план запроса. Выбор правильного плана, соответствующего структуре запроса и характеристикам данным, крайне важен для хорошей производительности, поэтому в системе работает сложный планировщик, задача которого — подобрать хороший план.

Узнать, какой план был выбран для какого-либо запроса, можно с помощью команды EXPLAIN.

Приведённые ниже примеры показаны на тестовой базе данных, которая создаётся для выявления регрессий в исходных кодах PostgreSQL текущей версии. Для неё предварительно выполняется VACUUM ANALYZE (очистка и анализ таблиц).

В этих примерах используется текстовый формат вывода EXPLAIN, принятый по умолчанию, как более компактный и удобный для восприятия человеком. Если вывод EXPLAIN нужно передать какой-либо программе для дальнейшего анализа, лучше использовать один из машинно-ориентированных форматов (XML, JSON или YAML).



План выполнения запроса, азы EXPLAIN

Структура плана запроса представляет собой дерево узлов плана.

Узлы на нижнем уровне дерева — это узлы сканирования, которые возвращают необработанные данные таблицы.

Разным типам доступа к таблице соответствуют разные узлы: последовательное сканирование, сканирование индекса и сканирование битовой карты.

Источниками строк могут быть не только таблицы, но и например, предложения VALUES и функции, возвращающие множества во FROM, и они представляются отдельными типами узлов сканирования.

Если запрос требует объединения, агрегатных вычислений, сортировки или других операций с исходными строками, над узлами сканирования появляются узлы, обозначающие эти операции.

И так как обычно операции могут выполняться разными способами, на этом уровне тоже могут быть узлы разных типов.



План выполнения запроса, азы EXPLAIN

В выводе команды EXPLAIN для каждого узла в дереве плана отводится одна строка, где показывается базовый тип узла плюс оценка стоимости выполнения данного узла, которую сделал для него планировщик.

Если для узла выводятся дополнительные свойства, в вывод могут добавляться дополнительные строки, с отступом от основной информации узла.

В самой первой строке (основной строке самого верхнего узла) выводится общая стоимость выполнения для всего плана; именно это значение планировщик старается минимизировать.



План выполнения запроса, азы EXPLAIN

Взгляните на следующий простейший пример, просто иллюстрирующий формат вывода:

```
EXPLAIN SELECT * FROM ticket_flights;
```

```
QUERY PLAN
```

```
-----
```

```
Seq Scan on ticket_flights (cost=0.00..43323.35 rows=2360335 width=32)
```

Этот запрос не содержит предложения WHERE, поэтому он должен просканировать все строки таблицы, так что планировщик выбрал план простого последовательного сканирования. Числа, перечисленные в скобках (слева направо), имеют следующий смысл:

Приблизительная стоимость запуска. Это время, которое проходит, прежде чем начнётся этап вывода данных, например для сортирующего узла это время сортировки.

Приблизительная общая стоимость. Она вычисляется в предположении, что узел плана выполняется до конца, то есть возвращает все доступные строки. На практике родительский узел может досрочно прекратить чтение строк дочернего (с LIMIT).

Ожидаемое число строк, которое должен вывести этот узел плана. При этом так же предполагается, что узел выполняется до конца.

Ожидаемый средний размер строк, выводимых этим узлом плана (в байтах).



План выполнения запроса, азы EXPLAIN

Стоимость может измеряться в произвольных единицах, определяемых параметрами планировщика:

seq_page_cost (floating point)

Задаёт приблизительную стоимость чтения одной страницы с диска, которое выполняется в серии последовательных чтений. Значение по умолчанию равно 1.0.

random_page_cost (floating point)

Задаёт приблизительную стоимость чтения одной произвольной страницы с диска. Значение по умолчанию равно 4.0.

cpu_tuple_cost (floating point)

Задаёт приблизительную стоимость обработки каждой строки при выполнении запроса. Значение по умолчанию — 0.01.

cpu_index_tuple_cost (floating point)

Задаёт приблизительную стоимость обработки каждой записи индекса при сканировании индекса. Значение по умолчанию — 0.005.

cpu_operator_cost (floating point)

Задаёт приблизительную стоимость обработки оператора или функции при выполнении запроса. Значение по умолчанию — 0.0025

И т.д.: <https://postgrespro.ru/docs/postgrespro/16/runtime-config-query#GUC-SEQ-PAGE-COST>



План выполнения запроса, азы EXPLAIN

Традиционно единицей стоимости считается операция чтения страницы с диска; то есть **seq_page_cost** (задаёт приблизительную стоимость чтения одной страницы с диска, которое выполняется в серии последовательных чтений) обычно равен 1.0, а другие параметры задаётся относительно него.

Важно понимать, что стоимость узла верхнего уровня включает стоимость всех его потомков. Также важно осознавать, что эта стоимость отражает только те факторы, которые учитывает планировщик.

В частности, она не зависит от времени, необходимого для передачи результирующих строк клиенту, хотя оно может составлять значительную часть общего времени выполнения запроса. Тем не менее планировщик игнорирует эту величину, так как он всё равно не сможет изменить её, выбрав другой план. (Мы верим в то, что любой правильный план запроса выдаёт один и тот же набор строк.)

Значение rows здесь имеет особенность — оно выражает не число строк, обработанных или просканированных узлом плана, а число строк, выданных этим узлом.

Часто оно окажется меньше числа просканированных строк в результате применённой к узлу фильтрации по условиям WHERE. В идеале, на верхнем уровне это значение будет приблизительно равно числу строк, которое фактически возвращает, изменяет или удаляет запрос.



План выполнения запроса, азы EXPLAIN

Возвращаясь к примеру:

```
EXPLAIN SELECT * FROM ticket_flights;
```

QUERY PLAN

Seq Scan on ticket_flights (cost=0.00..43323.35 rows=2360335 width=32)

Эти числа получаются очень просто. Выполните:

```
SELECT relpages, reltuples FROM pg_class WHERE relname = 'ticket_flights';
```

и вы увидите,

relpages	reltuples
integer	real
19720	2.360335e+06

что ticket_flights содержит **19720** страниц диска и **2360335** кортежей (т.е. строк).

Общая стоимость вычисляется как (число_чтений_диска * seq_page_cost) + (число_просканированных_строк * cpu_tuple_cost).

По умолчанию, seq_page_cost равно 1.0, а cpu_tuple_cost — 0.01, так что приблизительная стоимость запроса равна $(19720 * 1.0) + (2360335 * 0.01) = 43323.35$



План выполнения запроса, азы EXPLAIN

Теперь давайте изменим запрос, добавив в него предложение WHERE:

```
EXPLAIN SELECT * FROM ticket_flights WHERE amount > 7000;
```

QUERY PLAN	
text	
1	Seq Scan on ticket_flights (cost=0.00..49224.19 rows=1821277 width=32)
2	Filter: (amount > '7000'::numeric)

Заметьте, что в выводе EXPLAIN показано, что условие WHERE применено как «фильтр» к узлу плана Seq Scan (Последовательное сканирование). Это означает, что узел плана проверяет это условие для каждого просканированного им узла и выводит только те строки, которые удовлетворяют ему.

Предложение WHERE повлияло на оценку числа выходных строк. Однако при сканировании потребуется прочитать все 2360335 строк, поэтому общая стоимость не уменьшилась. На деле она даже немного увеличилась (на $2360335 * \text{cpu_operator_cost}$, если быть точными), отражая дополнительное время, которое потребуется процессору на проверку условия WHERE.

Фактическое число строк результата этого запроса будет равно 1821277, но значение rows даёт только приблизительное значение.



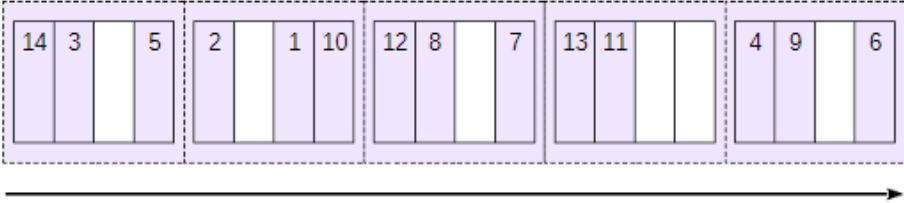
Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Seq Scan



Последовательное чтение всех страниц

- страницы читаются в кеш (используется буферное кольцо)
- проверяется видимость версий строк
- данные возвращаются в произвольном порядке
- время сканирования зависит от физического размера файла



3

В распоряжении оптимизатора имеется несколько способов доступа к данным. Самый простой из них — последовательное сканирование таблицы. Файл (или файлы) таблицы читается постранично от начала до конца. При этом рассматриваются все версии строк на каждой странице: удовлетворяют ли они условиям запроса и соблюдены ли правила видимости. Напомним, что чтение происходит через буферный кеш. Чтобы большая таблица не вытеснила все полезные данные, для последовательного сканирования создается «кольцо буферов» небольшого размера. При этом другие процессы, одновременно выполняющие последовательное сканирование той же таблицы, «присоединяются» к тому же кольцу и тем самым экономят дисковые чтения (если процесс присоединился не сразу, он затем отдельно дочитывает начальные страницы таблицы). Последовательное чтение файла позволяет использовать тот факт, что операционная система обычно читает данные порциями больше, чем размер страницы: с большой вероятностью несколько следующих страниц уже окажутся в кеше ОС.

Последовательное сканирование эффективно работает, когда надо прочитать всю таблицу или значительную ее часть (если селективность условия низка). Если же из всей таблицы нужна только небольшая часть записей, более предпочтительными являются методы доступа, использующие индекс.



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

В плане выполнения запроса последовательное сканирование представлено узлом Seq Scan:

```
=> EXPLAIN SELECT * FROM flights;
```

QUERY PLAN

```
-----  
Seq Scan on flights (cost=0.00..4772.67 rows=214867 width=63)  
(1 row)
```

В скобках приведены важные значения:

- cost — оценка стоимости;
- rows — оценка числа строк, возвращаемых операцией;
- width — оценка размера одной записи в байтах.

Стоимость указывается в некоторых условных единицах и состоит из двух компонент.

Первое число показывает начальную стоимость вычисления узла. Для последовательного сканирования это ноль — чтобы начать возвращать данные, никакой подготовки не требуется.

Второе число показывает полную стоимость для получения всех данных. Как оно получается?



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Оптимизатор PostgreSQL учитывает дисковый ввод-вывод и ресурсы процессора. Составляющая ввода-вывода рассчитывается как произведение числа страниц в таблице на условную стоимость чтения одной страницы:

```
=> SELECT relpages, current_setting('seq_page_cost'),
       relpages * current_setting('seq_page_cost')::real AS total
FROM pg_class WHERE relname='flights';
```

relpages	current_setting	total
2624	1	2624

(1 row)

Составляющая ресурсов процессора складывается из стоимости обработки каждой строки:

```
=> SELECT reltuples, current_setting('cpu_tuple_cost'),
       reltuples * current_setting('cpu_tuple_cost')::real AS total
FROM pg_class WHERE relname='flights';
```

reltuples	current_setting	total
214867	0.01	2148.67

(1 row)



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Агрегация

Рассмотрим более сложный запрос с агрегатной функцией (на небольшой таблице):

```
=> EXPLAIN SELECT count(*) FROM seats;
```

QUERY PLAN

```
-----  
Aggregate  (cost=24.74..24.75 rows=1 width=8)  
  -> Seq Scan on seats  (cost=0.00..21.39 rows=1339 width=0)  
(2 rows)
```

План состоит из двух узлов. Верхний — Aggregate, в котором происходит вычисление count, — получает данные от нижнего — Seq Scan.

Обратите внимание на стоимость узла Aggregate: начальная стоимость практически равна полной. Это означает, что узел не может выдать результат, пока не обработает все данные (что вполне логично).

Разница между оценкой для Aggregate и верхней оценкой для Seq Scan — стоимость работы собственно узла Aggregate. Она вычисляется исходя из оценки ресурсов на выполнение элементарной операции:

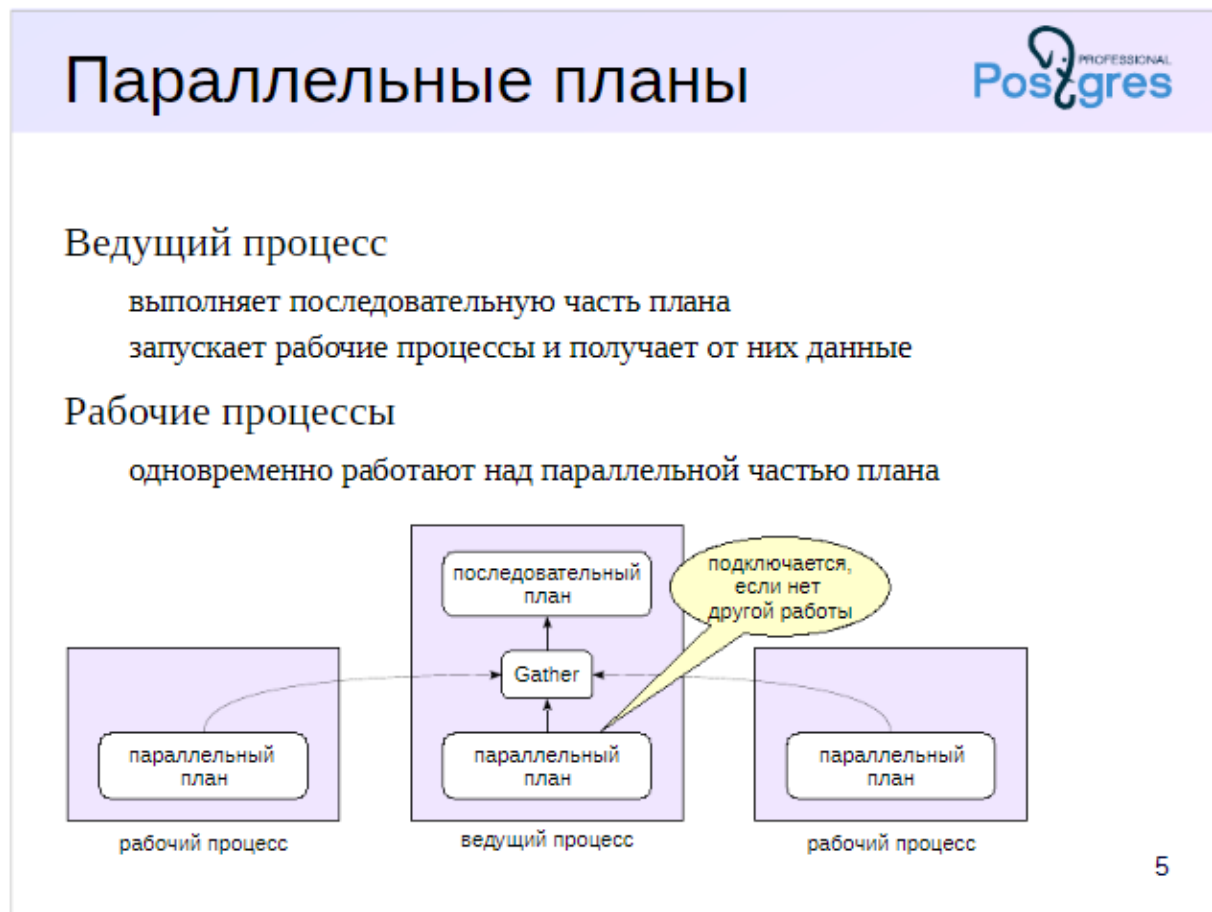
```
=> SELECT reltuples, current_setting('cpu_operator_cost'),  
       reltuples * current_setting('cpu_operator_cost')::real AS total  
FROM pg_class WHERE relname='seats';
```

reltuples	current_setting	total
1339	0.0025	3.3474998

(1 row)



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html



PostgreSQL поддерживает параллельное выполнение запросов.

Идея состоит в том, что ведущий процесс, выполняющий запрос, порождает (с помощью postmaster (postgres) - сервер баз данных) несколько рабочих процессов, которые одновременно выполняют одну и ту же «параллельную» часть плана.

Результаты этого выполнения передаются ведущему процессу, который собирает их в узле Gather.

Если рабочие процессы не успевают загрузить ведущего данными, ведущий процесс тоже подключается к выполнению того же самого параллельного плана.

Разумеется, запуск процессов и пересылка данных требуют определенных ресурсов, поэтому далеко не каждый запрос выполняется параллельно.

Кроме того, даже при параллельном выполнении не все шаги плана запроса могут быть распараллелены. Часть операций может выполняться ведущим процессом в одиночку, последовательно.

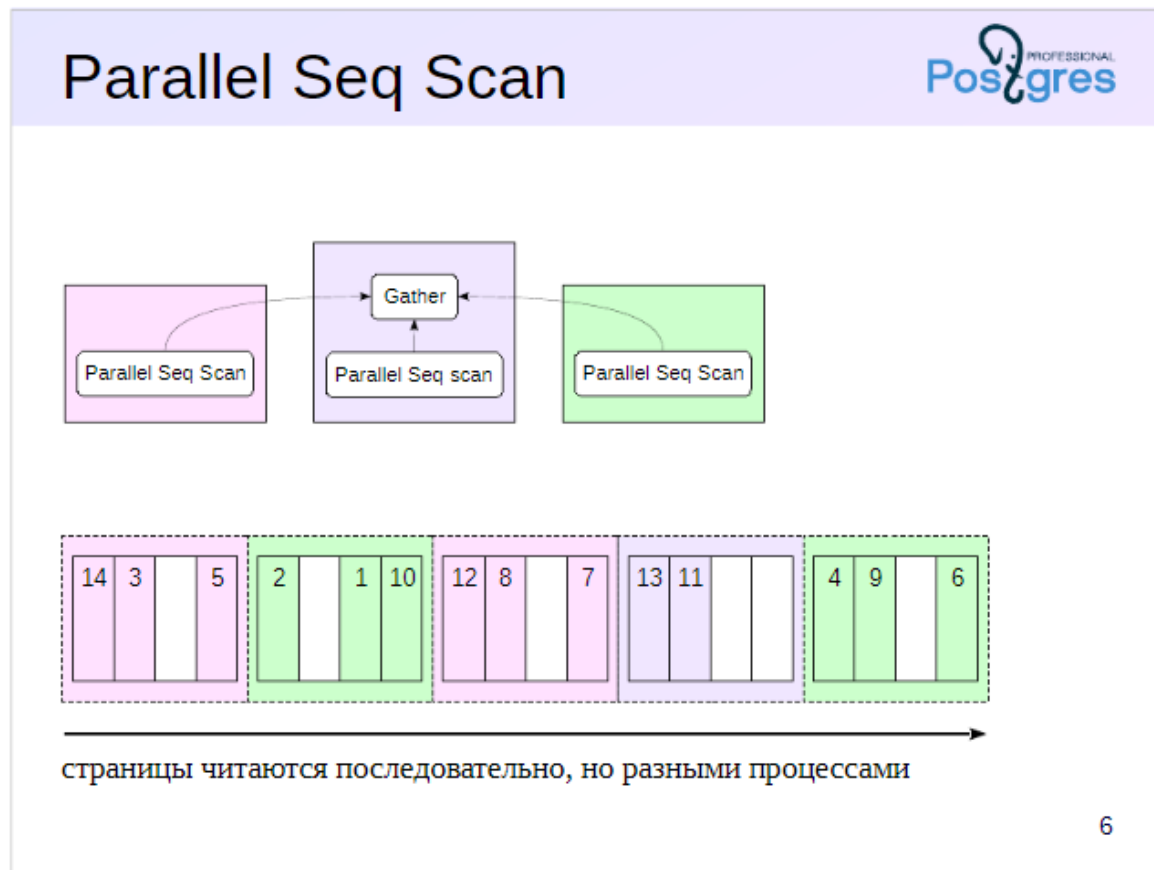
<https://postgrespro.ru/docs/postgresql/13/parallel-query>

Заметим, что в PostgreSQL нет другого теоретически возможного режима распараллеливания, при котором несколько процессов составляют конвейер для обработки данных (грубо говоря, отдельные узлы плана выполняются отдельными процессами).

Разработчики PostgreSQL сочли такой режим неэффективным.



Последовательное (полное) сканирование,
материал из курса «PostgreSQL 13. Оптимизация
запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html



Примером параллельного выполнения является Parallel Seq Scan — «параллельное последовательное сканирование».

Название звучит противоречиво, но, тем не менее, отражает суть операции.

Страницы таблицы читаются последовательно, в том же самом порядке, в котором они читались бы при обычном последовательном сканировании.

Однако запросы на чтение выдаются несколькими параллельно работающими процессами.

Процессы синхронизируются между собой, чтобы их запросы шли в правильном порядке.

Преимущество такого сканирования проявляется в том, что параллельные процессы одновременно обрабатывают свои страницы.

Чтобы эффект оправдал дополнительные накладные расходы на пересылку данных от процесса к процессу, обработка должна быть достаточно ресурсоемкой.

Хорошим примером является агрегация данных, поскольку для нее необходимы ресурсы процессора, а пересылать требуется только одно итоговое число.

В таком случае параллельное выполнение запроса может занять существенно меньше времени, чем последовательное.



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Параллельное последовательное сканирование

Рассмотрим пример плана с параллельным последовательным сканированием:

```
=> EXPLAIN SELECT count(*) FROM bookings;
```

QUERY PLAN

```
-----  
Finalize Aggregate  (cost=25442.58..25442.59 rows=1 width=8)  
  -> Gather  (cost=25442.36..25442.57 rows=2 width=8)  
      Workers Planned: 2  
      -> Partial Aggregate  (cost=24442.36..24442.37 rows=1 width=8)  
          -> Parallel Seq Scan on bookings  (cost=0.00..22243.29 rows=879629 width=0)  
(5 rows)
```

Все, что находится ниже узла Gather — параллельная часть плана. Она выполняется в каждом из рабочих процессов (которых запланировано два) и, возможно, в ведущем процессе.

Узел Gather и все узлы выше выполняются только в ведущем процессе. Это последовательная часть плана.



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Начнем разбираться снизу вверх. Узел Parallel Seq Scan представляет сканирование таблицы в параллельном режиме.

В поле rows показана оценка числа строк, которые обработает один рабочий процесс. Всего их запланировано 2, и еще часть работы выполнит ведущий, поэтому общее число строк делится на 2.4 (доля ведущего процесса уменьшается с ростом числа рабочих процессов).

```
=> SELECT round(reltuples / 2.4) "rows"  
FROM pg_class WHERE relname = 'bookings';
```

```
   rows  
-----  
  879629  
(1 row)
```

По умолчанию ведущий процесс участвует в выполнении параллельной части плана:

```
=> SHOW parallel_leader_participation;
```

```
parallel_leader_participation  
-----  
on  
(1 row)
```



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Если ведущий процесс становится узким местом, его можно разгрузить:

```
=> SET parallel_leader_participation = off;
```

SET

```
=> EXPLAIN SELECT count(*) FROM bookings;
```

QUERY PLAN

```
-----  
Finalize Aggregate  (cost=27641.65..27641.66 rows=1 width=8)  
  -> Gather  (cost=27641.44..27641.65 rows=2 width=8)  
        Workers Planned: 2  
          -> Partial Aggregate  (cost=26641.44..26641.45 rows=1 width=8)  
                -> Parallel Seq Scan on bookings  (cost=0.00..24002.55 rows=1055555 width=0)  
(5 rows)
```

В этом случае общее число строк делится на число запланированных рабочих процессов (2):

```
=> SELECT round(reltuples / 2) "rows"  
FROM pg_class WHERE relname = 'bookings';
```

```
rows  
-----  
1055555  
(1 row)
```



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Вернем значение по умолчанию для параметра `parallel_leader_participation`:

```
=> RESET parallel_leader_participation;
```

RESET

Для оценки узла Parallel Seq Scan компонента ввода-вывода берется полностью (таблицу все равно придется прочитать страница за страницей), а ресурсы процессора делятся между процессами (на 2.4 в данном случае).

```
=> SELECT round(  
    (  
        relpages * current_setting('seq_page_cost')::real +  
        reltuples * current_setting('cpu_tuple_cost')::real / 2.4  
    )::numeric,  
    2  
) "cost" FROM pg_class WHERE relname = 'bookings';
```

```
    cost  
-----  
 22243.29  
(1 row)
```



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

```
=> EXPLAIN SELECT count(*) FROM bookings;
```

QUERY PLAN

```
-----  
Finalize Aggregate (cost=25442.58..25442.59 rows=1 width=8)  
  -> Gather (cost=25442.36..25442.57 rows=2 width=8)  
        Workers Planned: 2  
        -> Partial Aggregate (cost=24442.36..24442.37 rows=1 width=8)  
              -> Parallel Seq Scan on bookings (cost=0.00..22243.29 rows=879629 width=0)  
(5 rows)
```

Следующий узел — Partial Aggregate — выполняет агрегацию данных, полученных рабочим процессом, то есть в данном случае подсчитывает количество строк. Оценка выполняется уже известным образом (и добавляется к оценке сканирования таблицы):

```
=> SELECT round((reltuples / 2.4 * current_setting('cpu_operator_cost')::real)::numeric, 2) "cost"  
FROM pg_class WHERE relname='bookings';
```

```
cost  
-----  
2199.07  
(1 row)
```



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Следующий узел — Gather — выполняется ведущим процессом. Он отвечает за запуск рабочих процессов и получение от них данных.

Запуск процессов и пересылка каждой строки данных оцениваются следующими значениями:

```
=> SELECT current_setting('parallel_setup_cost') parallel_setup_cost,  
current_setting('parallel_tuple_cost') parallel_tuple_cost;
```

parallel_setup_cost	parallel_tuple_cost
1000	0.1

(1 row)

В данном случае пересылается всего одна строка и основная стоимость приходится на запуск.

```
=> EXPLAIN SELECT count(*) FROM bookings;
```



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

QUERY PLAN

```
-----  
Finalize Aggregate (cost=25442.58..25442.59 rows=1 width=8)  
  -> Gather (cost=25442.36..25442.57 rows=2 width=8)  
        Workers Planned: 2  
          -> Partial Aggregate (cost=24442.36..24442.37 rows=1 width=8)  
                -> Parallel Seq Scan on bookings (cost=0.00..22243.29 rows=879629 width=0)  
(5 rows)
```

Последний узел — Finalize Aggregate — агрегирует полученные частичные агрегаты. Поскольку для этого надо сложить всего три числа, оценка минимальна.



Пример (https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html)

Практика



1. Убедитесь, что запрос, вычисляющий среднюю стоимость перелета, выполняется параллельно.
2. Проверьте, как выполняется тот же запрос, если чтение из таблицы перелетов оформлено как общее табличное выражение.
3. В демонстрации запрос всех строк из таблицы рейсов выполнялся последовательно. Почему? Может ли такой запрос в принципе быть распараллелен или он запрещает параллельное выполнение?
Воспользуйтесь параметром *force_parallel_mode*, чтобы ответить на этот вопрос.

15

2. Используйте указание **MATERIALIZED**, чтобы планировщик не раскрывал общее табличное выражение.

1. Средняя стоимость перелета

=> **EXPLAIN SELECT** avg(amount) **FROM** ticket_flights;

QUERY PLAN -----

Finalize Aggregate (cost=114640.79..114640.80 rows=1 width=32)

-> Gather (cost=114640.57..114640.77 rows=2 width=32)

Workers Planned: 2 ->

Partial Aggregate (cost=113640.57..113640.57 rows=1 width=32)

-> Parallel Seq Scan on ticket_flights (cost=0.00..104899.05 rows=3496605 width=6) (5 rows)

Планировщик выбирает параллельный план.

2. Общее табличное выражение

=> **EXPLAIN WITH** tf **AS MATERIALIZED** (**SELECT** * **FROM** ticket_flights) **SELECT** avg(amount) **FROM** tf;

QUERY PLAN -----

Aggregate (cost=342668.19..342668.20 rows=1 width=32) CTE tf

-> Seq Scan on ticket_flights (cost=0.00..153851.52 rows=8391852 width=32)

-> CTE Scan on tf (cost=0.00..167837.04 rows=8391852 width=16) (4 rows)

Здесь узел CTE представляет подзапрос в общем табличном выражении.

Затем выполняется узел CTE Scan: по сути он работает так же, как и Seq Scan, но просматривает не таблицу, а полученный и материализованный результат выполнения подзапроса. Эта операция выполняется последовательно.

При этом сам подзапрос в общем табличном выражении может работать параллельно:

=> **EXPLAIN WITH** x **AS MATERIALIZED** (**SELECT** avg(amount) **FROM** ticket_flights) **SELECT** * **FROM** x;

QUERY PLAN -----

CTE Scan on x (cost=114640.80..114640.82 rows=1 width=32) CTE x

-> Finalize Aggregate (cost=114640.79..114640.80 rows=1 width=32)

-> Gather (cost=114640.57..114640.77 rows=2 width=32)

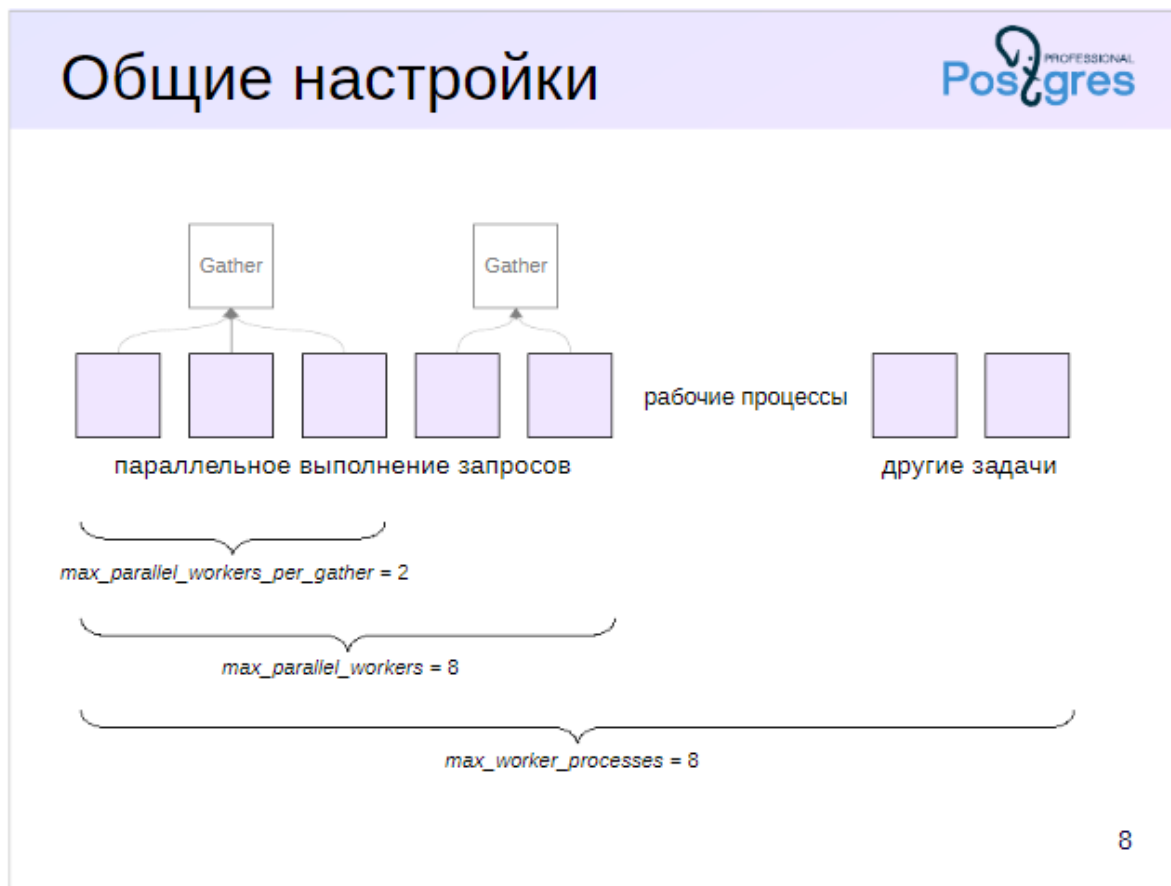
Workers Planned: 2

-> Partial Aggregate (cost=113640.57..113640.57 rows=1 width=32)

-> Parallel Seq Scan on ticket_flights (cost=0.00..104899.05 rows=3496605 width=6) (7 rows)



Последовательное (полное) сканирование,
материал из курса «PostgreSQL 13. Оптимизация
запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html



Параллельным выполнением управляет довольно много параметров. Сначала рассмотрим те, что ограничивают число рабочих процессов.

Вообще механизм рабочих процессов используется не только для параллельного выполнения запросов. Например, рабочие процессы задействованы в логической репликации, ими могут пользоваться расширения.

Общее число одновременно выполняющихся рабочих процессов ограничено параметром `max_worker_processes` (по умолчанию 8). Число одновременно выполняющихся рабочих процессов, занимающихся параллельными планами, ограничено параметром `max_parallel_workers` (по умолчанию тоже 8).

Число одновременно выполняющихся рабочих процессов, обслуживающих один ведущий процесс, ограничено параметром `max_parallel_workers_per_gather` (по умолчанию 2).

Значения этих параметров следует изменить в зависимости от возможностей аппаратуры, объема данных и загрузки системы.

Например, даже если в базе данных есть большие таблицы и запросы могли бы выиграть от распараллеливания, но при этом в системе нет свободных ядер, то параллельное выполнение не будет иметь смысла.



Последовательное (полное) сканирование,
материал из курса «PostgreSQL 13. Оптимизация
запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Число рабочих процессов



Равно нулю (параллельный план не строится)

если *размер таблицы* < *min_parallel_table_scan_size* = 8MB

Фиксировано

если для таблицы указан параметр хранения *parallel_workers*

Вычисляется по формуле

$1 + \lfloor \log_2(\text{размер таблицы} / \text{min_parallel_table_scan_size}) \rfloor$

Но не больше, чем *max_parallel_workers_per_gather*

9

Сколько рабочих процессов будет использоваться?

Планировщик вообще не будет рассматривать параллельное сканирование, если физический размер таблицы меньше значения параметра *min_parallel_table_scan_size*.

Обычно число процессов вычисляется по формуле, приведенной на слайде.

Она означает, что при увеличении таблицы в три раза будет добавляться очередной процесс.

Число процессов можно и явно указать в параметре хранения *parallel_workers* таблицы.

При этом число процессов в любом случае не будет превышать значения параметра *max_parallel_workers_per_gather*. Если при выполнении запроса доступное число процессов окажется меньше запланированного, будут использоваться только доступные (вплоть до последовательного выполнения, если пул полностью исчерпан).



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Число рабочих процессов

Планировщик не рассматривает параллельные планы для таблиц размером меньше, чем:

```
=> SHOW min_parallel_table_scan_size;
```

```
min_parallel_table_scan_size
-----
8MB
(1 row)
```

Если запросить информацию из таблицы немного большего размера (flights, 19 Мбайт), будет запланирован один дополнительный процесс:

```
=> EXPLAIN (analyze, costs off) SELECT count(*) FROM flights;
```



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

QUERY PLAN

```
-----  
Finalize Aggregate (actual time=67.734..70.208 rows=1 loops=1)  
  -> Gather (actual time=67.583..70.202 rows=2 loops=1)  
        Workers Planned: 1  
        Workers Launched: 1  
        -> Partial Aggregate (actual time=62.602..62.603 rows=1 loops=2)  
              -> Parallel Seq Scan on flights (actual time=3.524..57.289 rows=107434 loops=2)  
Planning Time: 0.094 ms  
Execution Time: 70.236 ms  
(8 rows)
```

При запросе данных из большой таблицы (bookings, 105 Мбайт) расчетное количество — три процесса.

=> **EXPLAIN** (analyze, costs off) **SELECT** count(*) **FROM** bookings;

QUERY PLAN

```
-----  
Finalize Aggregate (actual time=309.198..310.958 rows=1 loops=1)  
  -> Gather (actual time=307.887..310.950 rows=3 loops=1)  
        Workers Planned: 2  
        Workers Launched: 2  
        -> Partial Aggregate (actual time=304.727..304.728 rows=1 loops=3)  
              -> Parallel Seq Scan on bookings (actual time=4.354..201.346 rows=703703 loops=3)  
Planning Time: 0.049 ms  
Execution Time: 310.979 ms  
(8 rows)
```



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Однако запланировано два процесса, так как параметр `max_parallel_workers_per_gather` ограничивает число процессов параллельного участка плана и сейчас действует значение по умолчанию (2).

```
=> SHOW max_parallel_workers_per_gather;
```

```
max_parallel_workers_per_gather
-----
2
(1 row)
```

Ослабив ограничение, получим план с тремя процессами:

```
=> SET max_parallel_workers_per_gather = 5;
```

```
SET
```

```
=> EXPLAIN (analyze, costs off) SELECT count(*) FROM bookings;
```



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

QUERY PLAN

```
-----
Finalize Aggregate (actual time=208.378..214.335 rows=1 loops=1)
  -> Gather (actual time=206.314..214.329 rows=4 loops=1)
        Workers Planned: 3
        Workers Launched: 3
        -> Partial Aggregate (actual time=181.790..181.791 rows=1 loops=4)
              -> Parallel Seq Scan on bookings (actual time=0.022..114.372 rows=527778 loops=4)
Planning Time: 0.045 ms
Execution Time: 214.362 ms
(8 rows)
```

Для конкретной таблицы параметром хранения `parallel_workers` можно задать рекомендованное количество процессов:

```
=> ALTER TABLE bookings SET (parallel_workers = 4);
```

ALTER TABLE

```
=> EXPLAIN (analyze, costs off) SELECT count(*) FROM bookings;
```

QUERY PLAN

```
-----
Finalize Aggregate (actual time=214.461..216.738 rows=1 loops=1)
  -> Gather (actual time=212.575..216.731 rows=5 loops=1)
        Workers Planned: 4
        Workers Launched: 4
        -> Partial Aggregate (actual time=181.540..181.541 rows=1 loops=5)
              -> Parallel Seq Scan on bookings (actual time=0.022..84.888 rows=422222 loops=5)
Planning Time: 0.066 ms
Execution Time: 216.759 ms
(8 rows)
```



Последовательное (полное) сканирование, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Если установить параметр хранения в ноль, планировщик всегда будет выбирать последовательное сканирование данной таблицы.

Но в любом случае количество запланированных процессов не будет превышать `max_parallel_workers_per_gather`, независимо от параметра хранения.

Восстановим начальные значения параметров:

```
=> ALTER TABLE bookings RESET (parallel_workers);
```

```
ALTER TABLE
```

```
=> RESET max_parallel_workers_per_gather;
```

```
RESET
```



Последовательное (полное) сканирование,
материал из курса «PostgreSQL 13. Оптимизация
запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Не распараллеливаются



Запросы на запись
а также запросы с блокировкой строк

Курсоры
в том числе запросы в цикле FOR в PL/pgSQL

Запросы с функциями PARALLEL UNSAFE

Запросы в функциях,
вызванных из распараллеленного запроса

11

Не каждый запрос может выполняться в параллельном режиме. Не распараллеливаются запросы, изменяющие или блокирующие данные (UPDATE, DELETE, SELECT FOR UPDATE и т. п.).

Не распараллеливаются запросы, выполнение которых может быть приостановлено — это относится к запросам в курсорах, в том числе в циклах FOR PL/pgSQL.

Не распараллеливаются запросы, содержащие функции, помеченные как PARALLEL UNSAFE (все основные стандартные функции безопасны; список небезопасных можно получить из таблицы pg_proc по условию proparallel = 'u').

Не распараллеливаются запросы, содержащие функции, которые вызываются из распараллеленного запроса (чтобы не допустить рекурсивного разрастания).

Часть из этих ограничений может быть снята в следующих версиях PostgreSQL.

<https://postgrespro.ru/docs/postgresql/13/when-can-parallel-query-be-used>



Последовательное (полное) сканирование,
материал из курса «PostgreSQL 13. Оптимизация
запросов»: https://edu.postgrespro.ru/qpt-13/qpt_03_seqscan.html

Только последовательно



Чтение результатов общих табличных выражений (CTE)

Чтение результатов нераскрываемых подзапросов

Обращения к временным таблицам

Вызовы функций PARALLEL RESTRICTED

Функции, использующие вложенные транзакции

12

В целом, чем большую часть плана удастся выполнить параллельно, тем больший возможен эффект. Однако есть ряд операций, которые в целом не препятствуют распараллеливанию, но сами могут выполняться только последовательно в ведущем процессе.

К ним относятся:

- чтение результатов общих табличных выражений (подзапросов в предложении WITH);
- чтение результатов других нераскрываемых подзапросов (которые представляются в плане узлами, например, SubPlan);
- обращения ко временным таблицам (так как они доступны только ведущему процессу);
- вызовы функций, помеченных как PARALLEL RESTRICTED (список можно получить из таблицы pg_proc по условию proparallel = 'r').

Если в запросе вызывается функция, использующая вложенные транзакции (например, функции на PL/pgSQL с обработкой исключений), запрос завершится с ошибкой. Такие функции должны быть помечены как PARALLEL RESTRICTED.

<https://postgrespro.ru/docs/postgresql/13/parallel-safety>



Доступ на основе индексов

Все реляционные базы данных, в том числе PostgreSQL, позволяют создавать дополнительные, избыточные структуры, значительно ускоряя доступ к данным по сравнению с простым последовательным чтением.

Эти дополнительные структуры называются индексами.

Индексы - «избыточные» объекты базы данных; они не хранят никакой дополнительной информации, которую нельзя найти в исходной таблице.

Индексы предоставляют дополнительные пути доступа к данным; они позволяют определить, какие значения хранятся в строках таблицы, без необходимости чтения самой таблицы.

Если условие (или условия) фильтрации поддерживается индексом в таблице, индекс можно использовать для доступа к данным из этой таблицы.

Алгоритм извлекает список указателей на блоки, содержащие строки со значениями, удовлетворяющими условию фильтрации, и только эти блоки читаются из таблицы.



Доступ на основе индексов

Есть две отдельные физические операции, используемые PostgreSQL для получения строк с помощью индексов: индексное сканирование (index scan) и сканирование по битовой карте (bitmap heap scan).

При индексном сканировании движок базы данных считывает одну за другой все записи индекса, которые удовлетворяют условию фильтрации, и в этом же порядке извлекает блоки.

Так как базовая таблица представляет собой **кучу (heap)**, несколько записей индекса могут указывать на один и тот же блок.

Чтобы избежать многократного чтения одного и того же блока, в PostgreSQL реализована операция сканирования по битовой карте, которая создает битовую карту блоков, содержащих необходимые строки. Потом все строки в этих блоках фильтруются.

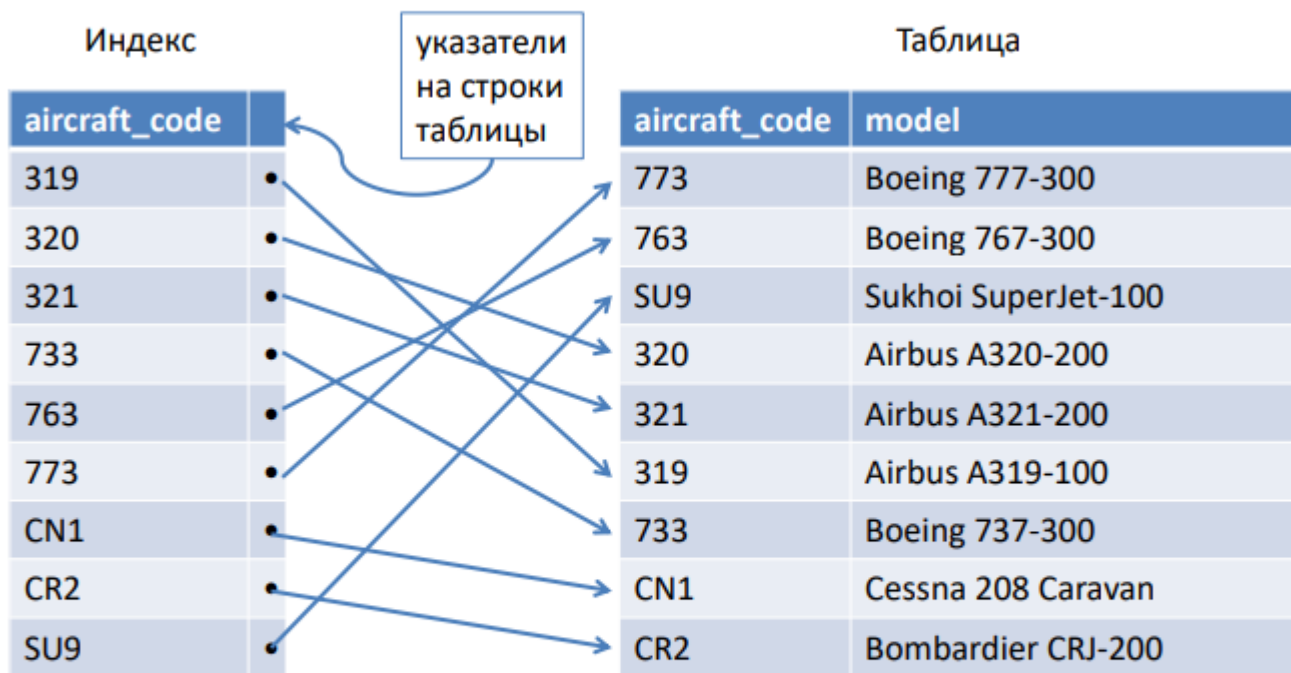
В PostgreSQL можно использовать нескольких индексов в одной и той же таблице в одном запросе, применяя логические операторы и/или к битовым картам блоков, порождаемым каждым индексом.



Использован материал презентации:

<https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Принципиальная схема индекса



- Записи в индексе *упорядочены*, а записи в таблице *не упорядочены*.
- При создании индекса значения атрибутов, участвующих в нем, могут комбинироваться и преобразовываться различными способами.



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Как используется индекс при поиске строк

- Специальная подсистема СУБД (планировщик) проверяет, *имеется ли для этой таблицы индекс*, созданный на основе тех же столбцов, что указаны, например, в условии предложения WHERE.
- Если такой индекс существует, то планировщик оценивает *целесообразность* его использования в данном конкретном случае.
- Если его использование целесообразно, то *сначала* выполняется поиск необходимых значений *в индексе*, а затем, если такие значения в нем найдены, производится обращение к таблице с использованием указателей, которые хранятся в записях индекса.
- Таким образом, *полный перебор* строк в таблице может быть заменен поиском *в упорядоченном индексе* и переходом к строке таблицы по прямому указателю (ссылке).

ВАЖНО! Индексы требуют некоторых накладных расходов на их создание и поддержание в актуальном состоянии при выполнении обновлений данных в таблицах.



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Создание индекса



```
CREATE INDEX имя_индекса ON имя_таблицы ( имя_столбца,  
... );
```

может быть более
одного столбца

можно не указывать

Пример. Создадим индекс для таблицы «Аэропорты» (airports) по столбцу airport_name.

```
CREATE INDEX ON airports ( airport_name );
```

```
CREATE INDEX
```

```
\d airports
```

```
...
```

Индексы:

```
...
```

```
"airports_airport_name_idx" btree (airport_name)
```

```
...
```

- Автоматически создаются индексы для ограничений PRIMARY KEY и UNIQUE.
- Индекс позволяет выполнять проверку на дублирование очень быстро.

суффикс



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Как увидеть созданные индексы?



`\d boarding_passes`

суффикс

тип индекса
B-дерево
(по умолчанию)

имена
столбцов

...

Индексы:

```
"boarding_passes_pkey" PRIMARY KEY, btree (ticket_no, flight_id)
"boarding_passes_flight_id_boarding_no_key" UNIQUE CONSTRAINT,
btree (flight_id, boarding_no)
"boarding_passes_flight_id_seat_no_key" UNIQUE CONSTRAINT, btree
(flight_id, seat_no)
```

...

Список всех индексов в текущей базе данных:

`\di` или `\di+`

вид ограничения,
для которого создан индекс

- Обратите внимание, что эти индексы созданы не по *одному* столбцу, а по *двум* столбцам.



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Проведем эксперимент:
выборка без индекса



Включим в утилите psql секундомер:
`\timing on`

Выключить можно будет так:
`\timing off`

```
SELECT count( * ) FROM tickets
WHERE passenger_name = 'IVAN IVANOV';
```

```
count
-----
```

```
200
(1 строка)
Время: 373,232 мс
```

Эти значения времени нужно рассматривать
лишь как качественные ориентиры.



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Проведем эксперимент: индекс создан



```
CREATE INDEX passenger_name  
ON tickets ( passenger_name );  
CREATE INDEX  
Время: 4023,408 мс
```

имя индекса

```
\d tickets  
...  
Индексы:  
...  
"passenger_name" btree (passenger_name)
```

```
SELECT count( * ) FROM tickets  
WHERE passenger_name = 'IVAN IVANOV';
```

```
count  
-----
```

```
200
```

```
(1 строка)
```

```
Время: 17,660 мс
```

время стало на порядок меньше



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Удаление индекса



Для удаления индекса используется команда:

```
DROP INDEX имя_индекса;
```

Давайте удалим созданный нами индекс для таблицы tickets:

```
DROP INDEX passenger_name;
```

```
DROP INDEX
```

Ряд замечаний



- Когда индекс уже создан, о его поддержании в актуальном состоянии заботится СУБД.
- Конечно, следует учитывать, что это требует от СУБД затрат ресурсов и времени.
- Индекс, созданный по столбцу, участвующему в соединении двух таблиц, может позволить ускорить процесс выборки записей из таблиц.
- При выборке записей в отсортированном порядке индекс также может помочь, если сортировка выполняется по тем столбцам, по которым индекс создан.



Использован материал презентации:

<https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Селективность индекса



- Если в SQL-запросе есть предложение ORDER BY, то индекс может позволить избежать этапа сортировки выбранных строк.
- Однако если SQL-запрос просматривает значительную часть таблицы, то *явная сортировка выбранных строк может оказаться быстрее*, чем использование индекса.
- Создавая индексы с целью ускорения доступа к данным, нужно учитывать предполагаемую долю строк таблицы (**селективность**), выбираемых при выполнении типичных запросов, в которых создаваемый индекс будет использоваться.
- Если эта *доля велика* (т. е. *селективность — низкая*), тогда наличие индекса может не дать ожидаемого эффекта.
- Индексы более полезны, когда из таблицы выбирается лишь небольшая доля строк, т. е. при высокой селективности выборки.



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Эксперимент с LIMIT и ORDER BY



Если для таблицы «Билеты» (tickets) еще не создан индекс по столбцу book_ref, то создайте его:

```
CREATE INDEX tickets_book_ref_test_key
ON tickets ( book_ref );
CREATE INDEX
SELECT * FROM tickets ORDER BY book_ref LIMIT 5;
...
```

Время: 0,442 мс

Удалите этот индекс и повторите запрос. Время его выполнения увеличится, вероятно, на два порядка. Почему?



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Порядок сортировки и значения NULL



- При создании индексов может использоваться не только возрастающий порядок значений в индексируемом столбце, но также и убывающий.
- По умолчанию порядок возрастающий (ASC), при этом значения NULL, которые также могут присутствовать в индексируемых столбцах, идут последними.

```
CREATE INDEX имя_индекса ON имя_таблицы  
( имя_столбца NULLS FIRST, ... );
```

значения NULL идут первыми

```
CREATE INDEX имя_индекса ON имя_таблицы  
( имя_столбца DESC NULLS LAST, ... );
```

убывающий порядок

значения NULL идут последними

Для разных столбцов можно указать разные порядки сортировки.



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Создание уникальных индексов



Индексы могут также использоваться для обеспечения уникальности значений атрибутов в строках таблицы. В таком случае создается уникальный индекс.

```
CREATE UNIQUE INDEX имя_индекса  
ON имя_таблицы ( имя_столбца, ... );
```

```
CREATE UNIQUE INDEX aircrafts_unique_model_key  
ON aircrafts ( model );
```

ПРИМЕЧАНИЕ. Мы могли при создании таблицы задать ограничение уникальности для столбца `model`, и тогда уникальный индекс был бы создан автоматически.

ВАЖНО! В уникальных индексах допускается наличие значений `NULL`, поскольку они считаются не совпадающими ни с какими другими значениями, в том числе и друг с другом.

Если уникальный индекс создан по нескольким атрибутам, то совпадающими считаются лишь те комбинации значений атрибутов в двух строках, в которых совпадают значения всех соответственных атрибутов.



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Использование функции при создании индекса



В команде создания индекса можно использовать не только имена столбцов, но также функции и скалярные выражения, построенные на основе столбцов таблицы.

Пример. Запретим значения столбца model в таблице aircrafts, отличающиеся только регистром символов.

```
CREATE UNIQUE INDEX aircrafts_unique_model_key  
ON aircrafts ( lower( model ) );
```

```
INSERT INTO aircrafts
```

```
VALUES ( '123', 'Cessna 208 CARAVAN', 1300 );
```

ОШИБКА: повторяющееся значение ключа нарушает ограничение уникальности, "aircrafts_unique_model_key"

ПОДРОБНОСТИ: Ключ "(lower(model))=(cessna 208 caravan)" уже существует.

такое значение хранится в индексе

перевод в нижний регистр

Индекс строится уже на основе преобразованных значений, поэтому при поиске строки в таблице искомое значение *сначала переводится в нижний регистр*, а затем осуществляется поиск в индексе.



Использован материал презентации:
<https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Ряд замечаний



- Индексы на основе выражений требуют *больше ресурсов* для их создания и поддержания при вставке и обновлении строк в таблице.
- Зато при выполнении выборок, построенных на основе сложных выражений, работа происходит с *меньшими накладными расходами*, поскольку в индексе хранятся уже *вычисленные значения* этих выражений, пусть даже самых сложных.
- Поэтому такие индексы целесообразно использовать тогда, когда *выборки производятся многократно*, и время, затраченное на создание и поддержание индекса, компенсируется (окупается) при выполнении выборок из таблицы.



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Правдоподобная ситуация



Частичный индекс формируется не для всех строк таблицы, а лишь для их *подмножества*. Это достигается с помощью использования условного выражения, называемого **предикатом индекса**. Предикат вводится с помощью предложения WHERE.

Пример. Представим, что руководство компании интересуют бронирования на сумму свыше одного миллиона рублей.

```
SELECT * FROM bookings
```

```
WHERE total_amount > 1000000]
```

```
ORDER BY book_date DESC;
```

предикат индекса

убывающий порядок

book_ref	book_date	total_amount
D7E9AA	2016-10-06 09:29:00+08	1062800.00
EF479E	2016-09-30 19:58:00+08	1035100.00
3AC131	2016-09-28 05:06:00+08	1087100.00
3B54BB	2016-09-02 21:08:00+08	1204500.00
65A6EA	2016-08-31 10:28:00+08	1065600.00

(5 строк)

Время: 90,996 мс



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Поможет ли частичный индекс?



```
CREATE INDEX bookings_book_date_part_key  
ON bookings ( book_date )  
WHERE total_amount > 1000000; }  
CREATE INDEX
```

поля book_date
нет в предикате

предикат индекса

- Хотя сортировка строк производится по датам бронирования в *убывающем* порядке, т. е. от более поздних дат к более ранним, тем не менее, включать ключевое слово *DESC* в индексное выражение, когда индекс создается только по одному столбцу, нет необходимости.
- Это объясняется тем, что PostgreSQL умеет совершать обход индекса как по *возрастанию*, так и по *убыванию* с одинаковой эффективностью.
- Обратите внимание, что индексируемый столбец *book_date* не участвует в формировании предиката индекса — в предикате используется столбец *total_amount*. Это вполне допустимая ситуация.

Повторим вышеприведенный запрос. Теперь он выдаст результат за время, *на порядок меньшее*, чем без использования частичного индекса.



Использован материал презентации: <https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-06.pdf>

Ряд замечаний



- Для того чтобы СУБД использовала частичный индекс, необходимо чтобы условие, записанное в запросе в предложении WHERE, соответствовало предикату индекса.
- Это означает, что либо условие должно быть точно таким же, как использованное в предикате частичного индекса при его создании, либо условие запроса должно математически сводиться к предикату индекса, а система должна суметь это понять.

Пример. В таком запросе ранее созданный индекс будет использоваться:

```
SELECT * FROM bookings WHERE total_amount > 1100000 ...
```

А в таком не будет:

подмножество строк

```
SELECT * FROM bookings WHERE total_amount > 900000 ...
```

- В большинстве случаев преимущества частичных индексов по сравнению с обычными индексами будут *минимальными*. Однако размер частичного индекса будет *меньше*, чем размер обычного. Для получения заметного полезного эффекта от их применения необходим опыт и понимание того, как работают индексы в PostgreSQL.



Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

В-дерево



Индекс

вспомогательная структура во внешней памяти
сопоставляет ключи и идентификаторы строк таблицы

Устройство: дерево поиска

сбалансированное
сильно ветвистое
только сортируемые типы данных (операции «больше», «меньше»)
результаты автоматически отсортированы

Использование

ускорение доступа
поддержка ограничений целостности

3

В этом модуле мы будем рассматривать только один из доступных в PostgreSQL типов индексов: В-дерево (B-tree). Это самый часто применяющийся на практике тип индекса. Некоторые другие типы индексов рассмотрены в курсе DEV2.

Как и все индексы в PostgreSQL, В-дерево является вторичной структурой — индекс не содержит в себе никакой информации, которую нельзя было бы получить из самой таблицы.

Индекс можно удалить и пересоздать. Индексы нужны для ускорения операций, затрагивающих небольшую часть таблицы: выборки малого количества строк, а также поддержки ограничений целостности (первичных и уникальных ключей).

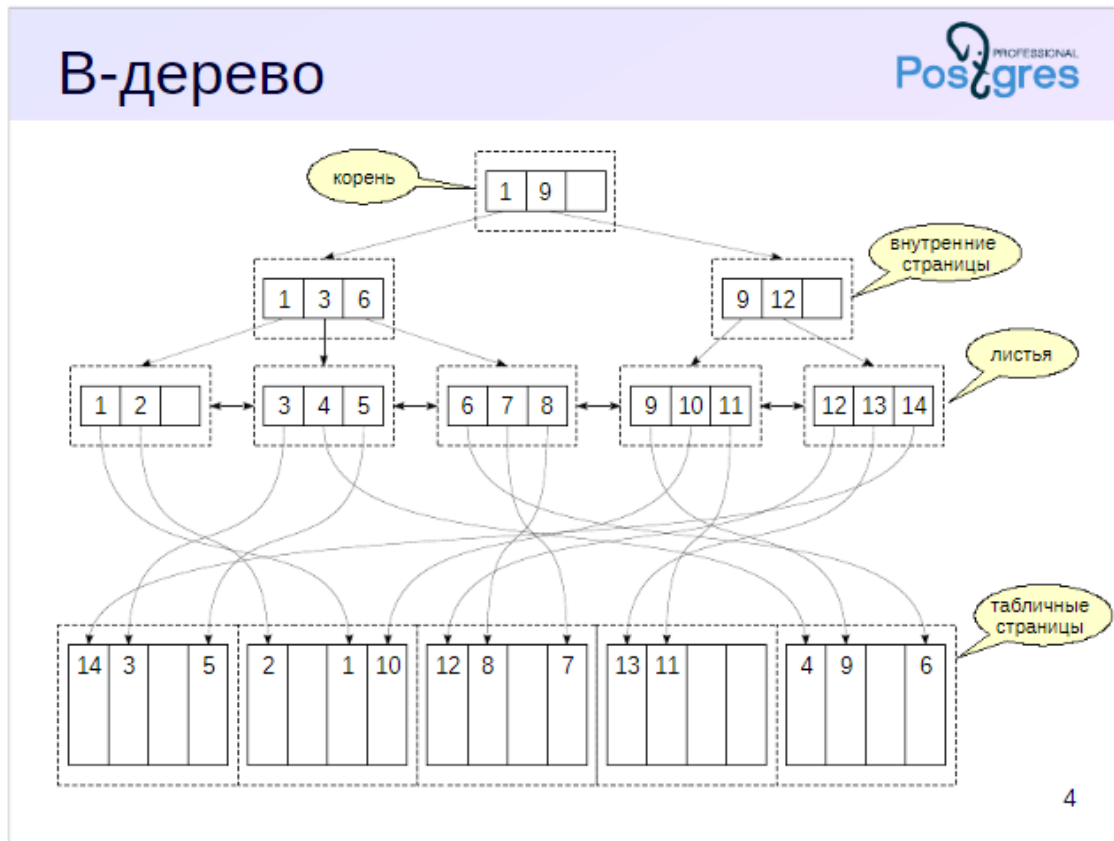
Любой индекс сопоставляет значения проиндексированных полей (ключи поиска) и идентификаторы строк таблицы. Для этого в индексе B-tree строится упорядоченное дерево ключей, в котором можно быстро найти нужный ключ, а вместе с ним — и ссылку на табличную строку.

Но проиндексировать можно только данные, допускающие сортировку (должны быть определены операции «больше», «меньше»). Например, числа, строки и даты могут быть проиндексированы, а точки на плоскости — нет (для них существуют другие типы индексов).

Особенностями В-дерева является его сбалансированность (постоянная глубина) и сильная ветвистость. Хотя размер дерева зависит от проиндексированных столбцов, на практике деревья обычно имеют глубину не больше 4-5.

Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html



На слайде представлен пример B-дерева (верхняя часть рисунка).

У дерева есть корневая, внутренние и листовые страницы.

Страницы состоят из индексных записей, каждая из которых содержит:

- значения столбцов, по которым построен индекс (такие столбцы называются ключевыми);
- ссылку на другую страницу индекса или версию строки.

Внутри страницы ключи всегда упорядочены.

Листовые страницы ссылаются непосредственно на табличные версии строк, содержащие ключи индексирования.

Внутренние страницы ссылаются на нижележащие страницы индекса, а значения ключей определяют диапазон значений, которые можно обнаружить, спустившись по ссылке.

Самая верхняя страница дерева, на которую нет ссылок, называется корнем.

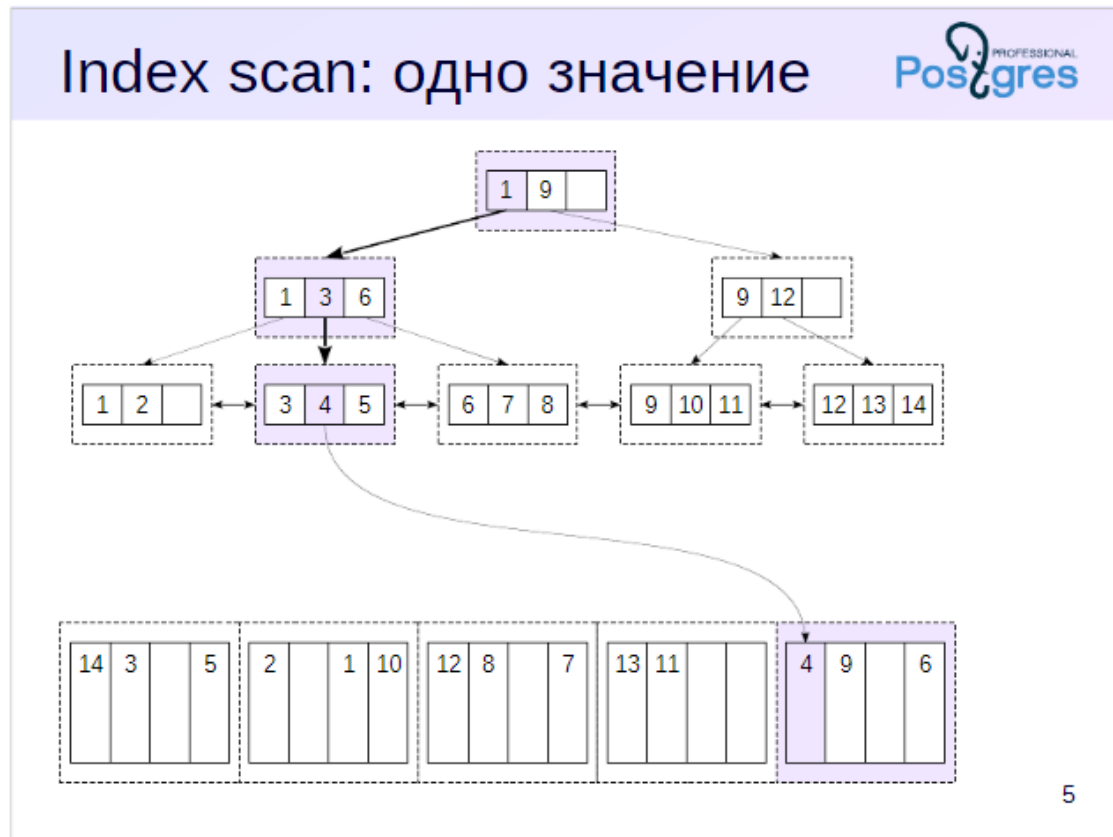
Индексная страница может быть заполнена не полностью. Свободное место используется для вставки в индекс новых значений.

Если же на странице не хватает места — она разделяется на две новых страницы. Разделившиеся страницы никогда не объединяются, и это в некоторых случаях может приводить к разрастанию индекса.



Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html



Рассмотрим поиск одного значения с помощью индекса. Например, нам нужно найти в таблице строку, где значение проиндексированного столбца равно четырем.

Начинаем с корня дерева. Индексные записи в корневой странице определяют диапазоны значений ключей в нижележащих страницах: «от 1 до 9» и «9 и больше».

Нам подходит диапазон «от 1 до 9», что соответствует строке с ключом 1.

Заметим, что ключи хранятся упорядоченно, следовательно, поиск внутри страницы выполняется очень эффективно.

Ссылка из найденной строки приводит нас к странице второго уровня. В ней мы находим диапазон «от 3 до 6» (ключ 3) и переходим к странице третьего уровня.

Эта страница является листовой. В ней мы находим ключ, равный 4, и переходим на страницу таблицы.

В действительности процесс более сложен: мы не говорили про неуникальные ключи, про проблемы одновременного доступа и так далее.

Но не будем углубляться в детали реализации.

Обратите внимание: на этой и следующих иллюстрациях цветом выделены те строки и те страницы, которые потребовалось прочитать.



Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Сканирование индекса

Рассмотрим таблицу бронирований:

=> \d bookings

Table "bookings.bookings"				
Column	Type	Collation	Nullable	Default
book_ref	character(6)		not null	
book_date	timestamp with time zone		not null	
total_amount	numeric(10,2)		not null	

Indexes:

"bookings_pkey" PRIMARY KEY, btree (book_ref)

Referenced by:

TABLE "tickets" CONSTRAINT "tickets_book_ref_fkey" FOREIGN KEY (book_ref) REFERENCES bookings(book_ref)

Столбец book_ref является первичным ключом и для него автоматически был создан индекс bookings_pkey.



Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Проверим план запроса с поиском одного значения:

```
=> EXPLAIN SELECT * FROM bookings WHERE book_ref = 'CDE08B';
```

QUERY PLAN

```
-----  
Index Scan using bookings_pkey on bookings (cost=0.43..8.45 rows=1 width=21)  
  Index Cond: (book_ref = 'CDE08B'::bpchar)  
(2 rows)
```

Выбран метод доступа Index Scan, указано имя использованного индекса. Здесь обращение и к индексу, и к таблице представлено одним узлом плана. Строкой ниже указано условие доступа.

Начальная стоимость индексного доступа — оценка ресурсов для спуска к листовому узлу. Она зависит от логарифма количества листовых узлов (количество операций сравнения, которые надо выполнить) и от высоты дерева. При оценке считается, что необходимые страницы окажутся в кеше, и оцениваются только ресурсы процессора: цифра получается небольшой.

Полная стоимость добавляет оценку чтения необходимых листовых страниц индекса и табличных страниц.



Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

В данном случае, поскольку индекс уникальный, будет прочитана одна индексная страница и одна табличная. Стоимость каждого из чтений оценивается параметром `random_page_cost`:

```
=> SELECT current_setting('random_page_cost');
```

```
current_setting
-----
4
(1 row)
```

Его значение обычно больше, чем `seq_page_cost`, поскольку произвольный доступ стоит дороже (хотя для SSD-дисков этот параметр следует существенно уменьшить).

Итого получаем 8, и еще немного добавляет оценка процессорного времени на обработку строк.

В строке `Index Cond` плана указываются только те условия, по которым происходит обращение к индексу или которые могут быть проверены на уровне индекса.



Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»: https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Дополнительные условия, которые можно проверить только по таблице, отображаются в отдельной строке Filter:

```
=> EXPLAIN SELECT * FROM bookings  
WHERE book_ref = 'CDE08B' AND total_amount > 1000;
```

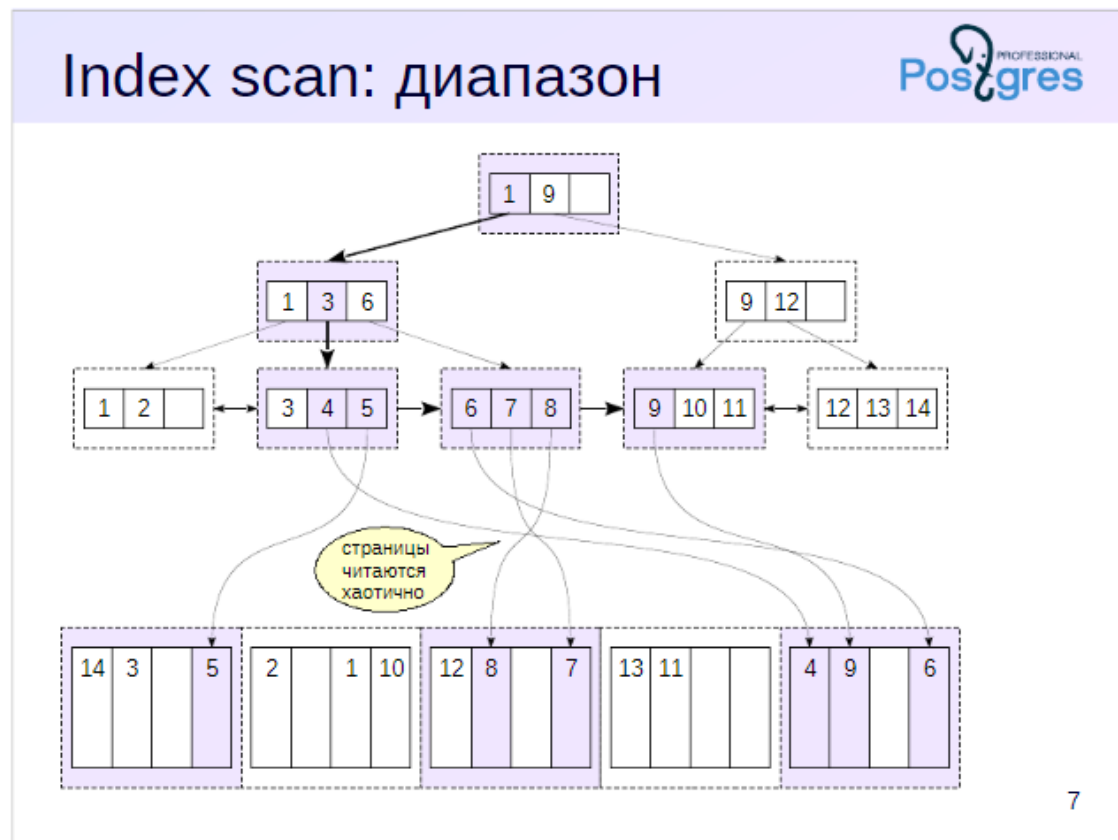
QUERY PLAN

```
-----  
Index Scan using bookings_pkey on bookings (cost=0.43..8.45 rows=1 width=21)  
  Index Cond: (book_ref = 'CDE08B'::bpchar)  
  Filter: (total_amount > '1000'::numeric)  
(3 rows)
```



Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html



B-дерево позволяет эффективно искать не только отдельные значения, но и диапазоны значений (по условиям «меньше», «больше», «меньше или равно», «больше или равно», а также «between»). Вот как это происходит.

Сначала мы ищем крайний ключ условия. Например, для условия «от 4 до 9» мы можем выбрать значение 4 или 9, а для условия «меньше 9» надо взять 9.

Затем спускаемся до листовой страницы индекса так, как мы рассматривали в предыдущем примере, и получаем первое значение из таблицы.

Дальше остается двигаться по листовым страницам индекса вправо (или влево, в зависимости от условия), перебирая строки этих страниц до тех пор, пока мы не встретим ключ, выпадающий из диапазона.

На слайде показан пример поиска значений по условию « x BETWEEN 4 AND 9» или, что тоже самое, « $x \geq 4$ AND $x \leq 9$ ». Спустившись к значению 4, мы затем перебираем ключи 5, 6, и так далее до 9.

Встретив ключ 10, прекращаем поиск. Нам помогают два свойства: упорядоченность ключей на всех страницах и связанность листовых страниц двунаправленным списком.

Обратите внимание, что к одной и той же табличной странице нам пришлось обращаться несколько раз.

Мы прочитали последнюю страницу таблицы (значение 4), затем первую (5), затем опять последнюю (6) и так далее.

Стоимость доступа на основе индексов

Стоимостная модель этого алгоритма намного сложнее.

Неформально ее можно описать так: при малых значениях селективности, скорее всего, все строки, удовлетворяющие условиям фильтрации, будут располагаться в разных блоках, и, следовательно, стоимость будет пропорциональна количеству возвращаемых строк.

Для больших значений селективности количество обрабатываемых блоков приближается к общему количеству блоков.

В последнем случае стоимость становится выше, чем стоимость полного сканирования, поскольку для доступа к индексу необходимы ресурсы.



Поиск по диапазону https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Мы получаем данные из индекса, спускаясь от корня дерева к левому листовому узлу и проходя по списку листовых страниц. Поэтому индексное сканирование всегда возвращает данные в том порядке, в котором они хранятся в дереве индекса и который был указан при его создании:

```
=> EXPLAIN (costs off) SELECT * FROM bookings  
WHERE book_ref > '000900' AND book_ref < '000939'  
ORDER BY book_ref;
```

QUERY PLAN

Index Scan using bookings_pkey on bookings

Index Cond: ((book_ref > '000900'::bpchar) AND (book_ref < '000939'::bpchar))

(2 rows)



Поиск по диапазону https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Тот же самый индекс может использоваться и для получения строк в обратном порядке:

```
=> EXPLAIN (costs off) SELECT * FROM bookings  
WHERE book_ref > '000900' AND book_ref < '000939'  
ORDER BY book_ref DESC;
```

QUERY PLAN

Index Scan Backward using bookings_pkey on bookings

Index Cond: ((book_ref > '000900'::bpchar) AND (book_ref < '000939'::bpchar))

(2 rows)

В этом случае мы спускаемся от корня дерева к правому листовому узлу, и проходим по списку листовых страниц в обратную сторону.



Поиск по диапазону https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Обратите внимание на количество страниц (Buffers), которое потребовалось прочитать:

=> EXPLAIN (analyze, buffers, costs off, timing off, summary off)

```
SELECT * FROM bookings
```

```
WHERE book_ref > '000900' AND book_ref < '000939'
```

```
ORDER BY book_ref DESC;
```

QUERY PLAN

Index Scan Backward using bookings_pkey on bookings (actual rows=5 loops=1)

Index Cond: ((book_ref > '000900'::bpchar) AND (book_ref < '000939'::bpchar))

Buffers: shared hit=4 read=1

Planning:

Buffers: shared hit=8

(5 rows)



Поиск по диапазону https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Сравним поиск по диапазону с повторяющимся поиском отдельных значений.
Получим тот же результат с помощью конструкции IN, и посмотрим, сколько страниц потребовалось прочитать в этом случае:

```
=> EXPLAIN (analyze, buffers, costs off)
SELECT * FROM bookings
WHERE book_ref IN ('000906','000909','000917','000930','000938')
ORDER BY book_ref DESC;
```

QUERY PLAN

Index Scan Backward using bookings_pkey on bookings (actual time=0.026..0.039 rows=5 loops=1)

Index Cond: (book_ref = ANY ('{000906,000909,000917,000930,000938}'::bpchar[]))

Buffers: shared hit=24

Planning Time: 0.082 ms

Execution Time: 0.047 ms

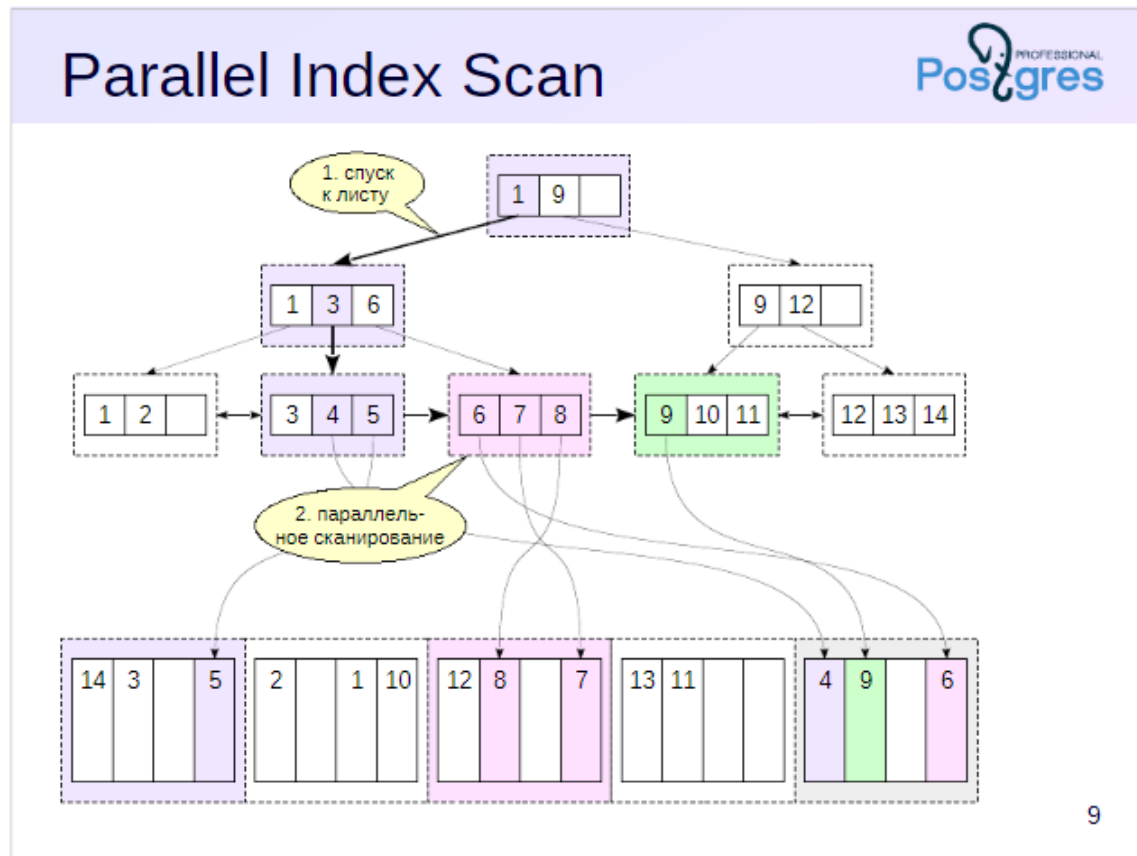
(5 rows)

Количество страниц увеличилось, поскольку в этом случае приходится спускаться от корня к каждому значению.



Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html



Индексный доступ может выполняться в параллельном режиме. Это происходит в два этапа.

Сначала ведущий процесс спускается от корня дерева к листовой странице.

Затем рабочие процессы выполняют параллельное чтение листовых страниц индекса, двигаясь по указателям.

Процесс, прочитавший индексную страницу, выполняет и чтение необходимых табличных страниц.

При этом может получиться так, что одну и ту же табличную страницу прочитают несколько процессов (этот пример показан на иллюстрации: последняя табличная страница содержит строки, ссылки на которые ведут от нескольких индексных страниц, прочитанных разными процессами).

Конечно, сама страница будет находиться в буферном кеше в одном экземпляре.



Параллельное сканирование индекса

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Сканирование индекса может выполняться в параллельном режиме. В качестве примера найдем общую сумму всех бронирований с номерами, меньшими 400000 (их примерно одна четверть от общего числа):

```
=> EXPLAIN SELECT sum(total_amount)
```

```
FROM bookings WHERE book_ref < '400000';
```

QUERY PLAN

```
-----  
Finalize Aggregate (cost=16864.79..16864.80 rows=1 width=32)  
  -> Gather (cost=16864.57..16864.78 rows=2 width=32)  
      Workers Planned: 2  
      -> Partial Aggregate (cost=15864.57..15864.58 rows=1 width=32)  
          -> Parallel Index Scan using bookings_pkey on bookings (cost=0.43..15314.81  
rows=219906 width=6)  
              Index Cond: (book_ref < '400000'::bpchar)  
(6 rows)
```

Аналогичный план мы уже видели при параллельном последовательном сканировании, но в данном случае данные читаются с помощью индекса — узел Parallel Index Scan.



Параллельное сканирование индекса

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Полная стоимость складывается из стоимостей доступа к таблице и к индексу. Стоимость доступа к 1/4 табличных страниц (поделенных между процессами) оценивается аналогично последовательному сканированию:

```
=> SELECT round(  
    (relpages / 4.0) * current_setting('seq_page_cost')::real +  
    (reltuples / 4.0) / 2.4 * current_setting('cpu_tuple_cost')::real  
) FROM pg_class WHERE relname = 'bookings';
```

round

5561

(1 row)



Параллельное сканирование индекса

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Оценка стоимости индексного доступа не делится между процессами, так как индекс читается процессами последовательно, страница за страницей:

```
=> SELECT round(  
    (relpages / 4.0) * current_setting('random_page_cost')::real +  
    (reltuples / 4.0) * current_setting('cpu_index_tuple_cost')::real +  
    (reltuples / 4.0) * current_setting('cpu_operator_cost')::real  
) FROM pg_class WHERE relname = 'bookings_pkey';
```

round

9750

(1 row)

Значением параметра `cpu_operator_cost` оценивается операция сравнения значений («меньше»).



Индексный доступ, материал из курса
«PostgreSQL 13. Оптимизация запросов»:
https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Число рабочих процессов



Равно нулю (параллельный план не строится)

если *размер выборки* < *min_parallel_index_scan_size* = 512kB

Фиксировано

если для таблицы указан параметр хранения *parallel_workers*

Вычисляется по формуле

$1 + \lfloor \log_3(\text{размер выборки} / \text{min_parallel_index_scan_size}) \rfloor$

Но не больше, чем *max_parallel_workers_per_gather*

11

Число рабочих процессов выбирается примерно так же, как и в случае последовательного сканирования.

Сравнивается объем данных, который предполагается прочитать из индекса (определяемый числом индексных страниц) со значением параметра *min_parallel_index_scan_size* (по умолчанию 512kB).

Если в случае последовательного сканирования таблицы объем данных определялся размером всей таблицы, то при индексном доступе планировщик должен спрогнозировать, сколько индексных страниц будет прочитано.

Как именно это происходит, рассматривается позже в теме «Статистика».

Если размер выборки меньше, параллельный план не рассматривается оптимизатором.

Например, никогда не будет выполняться параллельно доступ к одному значению — в этом случае просто нечего распараллеливать.

Если размер выборки достаточно велик, число рабочих процессов определяется по формуле, если только оно не указано явно в параметре хранения *parallel_workers* таблицы (не индекса!).

Число процессов в любом случае не будет превышать значения параметра *max_parallel_workers_per_gather*.



Число рабочих процессов https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Установим максимальное число параллельно работающих процессов, которые могут запускаться одним узлом Gather, в значение четыре:

```
=> SET max_parallel_workers_per_gather = 4;
```

SET

Посмотрим значение конфигурационного параметра min_parallel_index_scan_size:

```
=> SHOW min_parallel_index_scan_size;
```

```
min_parallel_index_scan_size
```

```
-----
```

```
512kB
```

```
(1 row)
```



Число рабочих процессов https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

И выполним следующий запрос:

```
=> EXPLAIN SELECT sum(total_amount)
FROM bookings WHERE book_ref < '400000';
QUERY PLAN
```

```
-----
Finalize Aggregate (cost=16244.20..16244.21 rows=1 width=32)
-> Gather (cost=16243.87..16244.18 rows=3 width=32)
    Workers Planned: 3
        -> Partial Aggregate (cost=15243.87..15243.88 rows=1 width=32)
            -> Parallel Index Scan using bookings_pkey on bookings (cost=0.43..14818.24
rows=170250 width=6)
                Index Cond: (book_ref < '400000'::bpchar)
(6 rows)
```

Было запланировано три параллельных процесса. Если увеличить значение `min_parallel_index_scan_size` до 10 мегабайт, планировщик запланирует лишь один процесс:

```
=> SET min_parallel_index_scan_size = '10MB';
```

```
SET
```



Число рабочих процессов https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

И повторим запрос:

```
=> EXPLAIN SELECT sum(total_amount)
FROM bookings WHERE book_ref < '400000';
```

QUERY PLAN

Finalize Aggregate (cost=17996.56..17996.57 rows=1 width=32)

-> Gather (cost=17996.44..17996.55 rows=1 width=32)

Workers Planned: 1

-> Partial Aggregate (cost=16996.44..16996.45 rows=1 width=32)

-> Parallel Index Scan using bookings_pkey on bookings (cost=0.43..16220.30
rows=310456 width=6)

Index Cond: (book_ref < '400000'::bpchar)

(6 rows)

```
=> RESET min_parallel_index_scan_size;
```

RESET

```
=> RESET max_parallel_workers_per_gather;
```

RESET



Сканирование только индекса

Операции доступа к данным не обязательно возвращают полные строки.

Если некоторые столбцы не нужны для запроса, их можно опустить, как только строка пройдет условия фильтрации (если таковые имеются). Говоря более формально, это означает, что логическая проекция сочетается с доступом к данным. Такое сочетание особенно полезно, если индекс, используемый для фильтрации, содержит все столбцы, необходимые для запроса.

Алгоритм считывает данные из индекса и применяет оставшиеся условия фильтрации, если это необходимо.

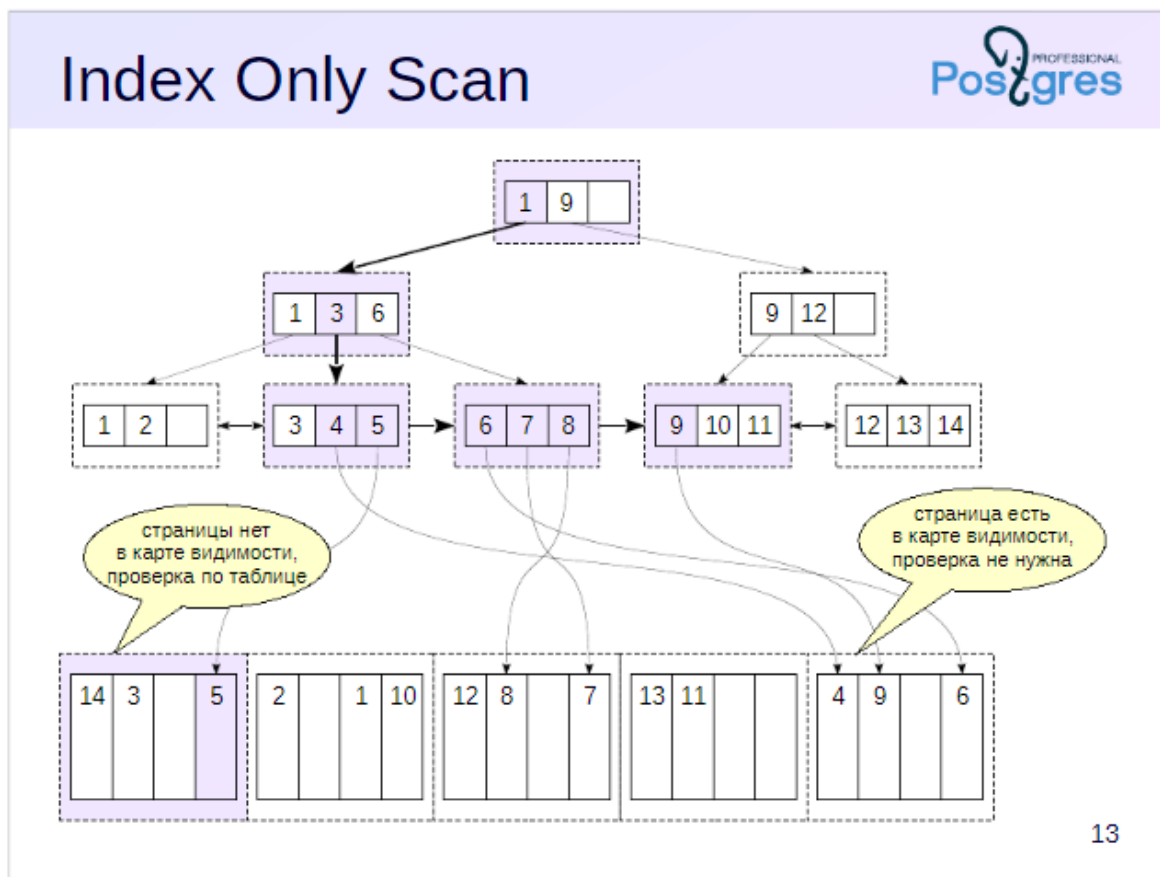
Стоимостная модель сканирования только индекса аналогична модели для доступа к таблице на основе индекса, за исключением того, что не нужно обращаться к данным таблицы (не всегда).

Для малых значений селективности стоимость примерно пропорциональна количеству возвращаемых строк. При больших значениях селективности алгоритм выполняет (почти) полный просмотр индекса. Стоимость просмотра индекса обычно ниже, чем стоимость полного просмотра таблицы, потому что индекс содержит меньше данных.



Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html



Если в запросе требуются только проиндексированные данные, то они уже есть в самом индексе — к таблице в этом случае обращаться не надо.

Такой индекс называется покрывающим для запроса.

Это хорошая оптимизация, исключающая обращения к табличным страницам.

Но, к сожалению, индексные страницы не содержат информацию о видимости строк — чтобы проверить, надо ли показывать найденную в индексе строку, мы вынуждены заглянуть и в табличную страницу, что сводит оптимизацию на нет.

Поэтому критическую роль в эффективности сканирования только индекса играет карта видимости. Если табличная страница содержит гарантированно видимые данные, и это отражено в карте видимости, к такой табличной странице обращаться не надо. Но страницы, не отмеченные в карте видимости, посетить все-таки придется.

Это одна из причин, по которой стоит выполнять очистку (vacuum) достаточно часто — именно этот процесс обновляет карту видимости.

Планировщик не знает точно, сколько табличных страниц потребуют проверки, но учитывает оценку этого числа. При плохом прогнозе планировщик может отказаться от использования сканирования только индекса.

При обработке каждой индексной записи сначала проверяется наличие табличной страницы в карте видимости, а при ее отсутствии читается сама табличная страница.

Сканирование только индекса https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Если вся необходимая информация содержится в самом индексе, то нет необходимости обращаться к таблице — за исключением проверки видимости:

```
=> EXPLAIN SELECT book_ref FROM bookings WHERE book_ref <= '100000';
```

QUERY PLAN

Index Only Scan using bookings_pkey on bookings (cost=0.43..4199.48 rows=147374 width=7)
Index Cond: (book_ref <= '100000'::bpchar)
(2 rows)

Посмотрим план этого запроса с помощью EXPLAIN ANALYZE:

```
=> EXPLAIN (analyze, costs off, timing off, summary off)  
SELECT book_ref FROM bookings WHERE book_ref <= '100000';
```

QUERY PLAN

Index Only Scan using bookings_pkey on bookings (actual rows=132109 loops=1)
Index Cond: (book_ref <= '100000'::bpchar)
Heap Fetches: 0
(3 rows)

Строка Heap Fetches показывает, сколько версий строк было проверено с помощью таблицы. В данном случае карта видимости содержит актуальную информацию, обращаться к таблице не потребовалось.



Сканирование только индекса https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Обновим первую строку таблицы:

```
=> UPDATE bookings  
SET total_amount = total_amount  
WHERE book_ref = '000004';  
UPDATE 1
```

Сколько версий строк придется проверить теперь?

```
=> EXPLAIN (analyze, costs off, timing off, summary off)  
SELECT book_ref FROM bookings WHERE book_ref <= '100000';
```

QUERY PLAN

```
-----  
Index Only Scan using bookings_pkey on bookings (actual rows=132109 loops=1)  
  Index Cond: (book_ref <= '100000'::bpchar)  
  Heap Fetches: 158  
(3 rows)
```

Проверять приходится все версии, попадающие на измененную страницу.



Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Include-индексы



Индекс с дополнительными неключевыми столбцами

```
CREATE INDEX ... INCLUDE (...)
```

Неключевые столбцы

- не используются при поиске по индексу
- не учитываются ограничением уникальности
- значения хранятся в индексной записи
- и возвращаются без обращения к таблице

15

Покрывающий индекс как правило увеличивает эффективность выборки.

Чтобы сделать индекс покрывающим, в него может понадобиться добавить столбцы, но это не всегда возможно:

- добавление столбца в уникальный индекс нарушит гарантию уникальности исходных столбцов;
- тип данных добавляемого столбца может не поддерживаться индексом.

В таких случаях начиная с версии PostgreSQL 11 можно добавить к индексу неключевые столбцы, указав их в предложении INCLUDE.

<https://postgrespro.ru/docs/postgresql/13/sql-createindex>

Значения таких столбцов не формируют дерево индекса, а просто хранятся как дополнительные сведения в индексных записях листовых страниц.

Поиск по неключевым столбцам не работает, но их значения могут возвращаться без обращения к таблице.

В настоящее время include-индексы поддерживаются только для индексов на основе B-деревьев и для GiST-индексов.

Include-индексы создаются, чтобы индекс стал покрывающим, но не стоит путать эти два термина. Индекс вполне может быть покрывающим для некоторого запроса, но не использовать предложение INCLUDE.

А Include-индекс может не быть покрывающим для каких-то запросов.



Include-индексы https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Индекс tickets_pkey не является покрывающим для приведенного запроса, поскольку в нем требуется не только столбец ticket_no (есть в индексе), но и book_ref:

=> EXPLAIN (analyze, buffers, costs off, summary off)

SELECT ticket_no, book_ref FROM tickets WHERE ticket_no > '0005435990286';

QUERY PLAN

Index Scan using tickets_pkey on tickets (actual time=1.037..7.135 rows=7146 loops=1)
 Index Cond: (ticket_no > '0005435990286'::bpchar)
 Buffers: shared hit=72 read=150
Planning:
 Buffers: shared hit=19 read=7
(5 rows)

В данном случае было прочитано 222 страницы (Buffers).



Include-индексы https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Создадим include-индекс, добавив в него неключевой столбец book_ref, так как он требуется запросу:

```
=> CREATE UNIQUE INDEX ON tickets (ticket_no) INCLUDE (book_ref);
```

CREATE INDEX

Повторим запрос:

```
=> EXPLAIN (analyze, buffers, costs off, summary off)
SELECT ticket_no, book_ref FROM tickets WHERE ticket_no > '0005435990286';
```

QUERY PLAN

Index Only Scan using tickets_ticket_no_book_ref_idx on tickets (actual time=0.054..2.078 rows=7146 loops=1)

Index Cond: (ticket_no > '0005435990286'::bpchar)

Heap Fetches: 0

Buffers: shared hit=4 read=35

Planning:

Buffers: shared hit=19 read=4
(6 rows)

Теперь оптимизатор выбирает метод Index Only Scan и использует только что созданный индекс. Количество прочитанных страниц сократилось. Поскольку карта видимости актуальна, обращаться к таблице не пришлось (Heap Fetches: 0).

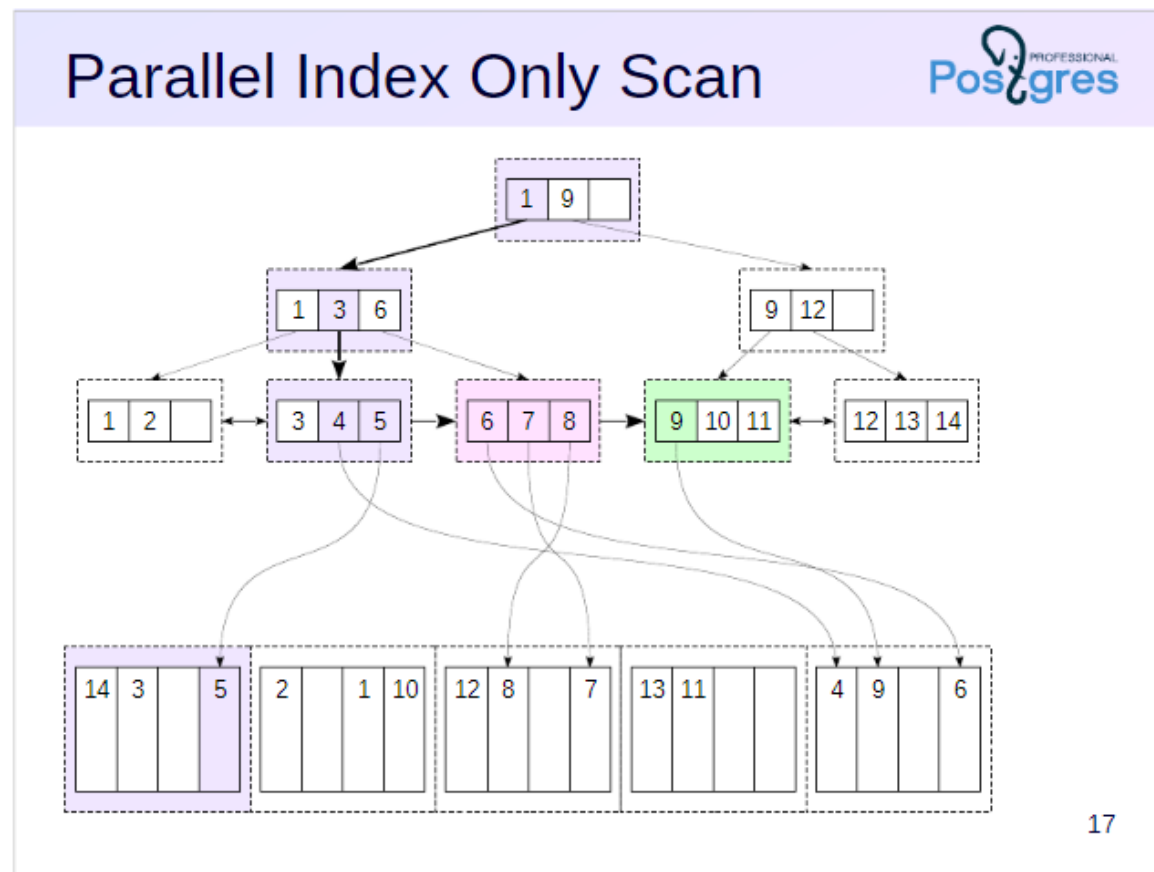
В include-индекс можно включать столбцы с типами данных, которые не поддерживаются B-деревом (например, геометрические типы и xml).



Индексный доступ, материал из курса «PostgreSQL 13.

Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html



Сканирование только индекса может выполняться параллельно. Это происходит точно так же, как и при обычном индексном сканировании: ведущий процесс спускается от корня к листовой странице, а затем рабочие процессы параллельно сканируют листовые страницы индекса, обращаясь при необходимости к соответствующим страницам таблицы для проверки видимости.

Параллельное сканирование только индекса

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Сканирование только индекса также может выполняться параллельно:

```
=> EXPLAIN SELECT count(book_ref)
FROM bookings WHERE book_ref <= '400000';
```

QUERY PLAN

```
-----
Finalize Aggregate (cost=13499.80..13499.81 rows=1 width=8)
  -> Gather (cost=13499.58..13499.79 rows=2 width=8)
        Workers Planned: 2
          -> Partial Aggregate (cost=12499.58..12499.59 rows=1 width=8)
                -> Parallel Index Only Scan using bookings_pkey on bookings (cost=0.43..11949.82
rows=219907 width=7)
                    Index Cond: (book_ref <= '400000'::bpchar)
(6 rows)
```



Параллельное сканирование только индекса

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Стоимость доступа к таблице здесь учитывает только обработку строк, без ввода-вывода:

```
=> SELECT round(  
    (reltuples / 4.0) / 2.4 * current_setting('cpu_tuple_cost')::real  
) FROM pg_class WHERE relname = 'bookings';
```

round

2199

(1 row)

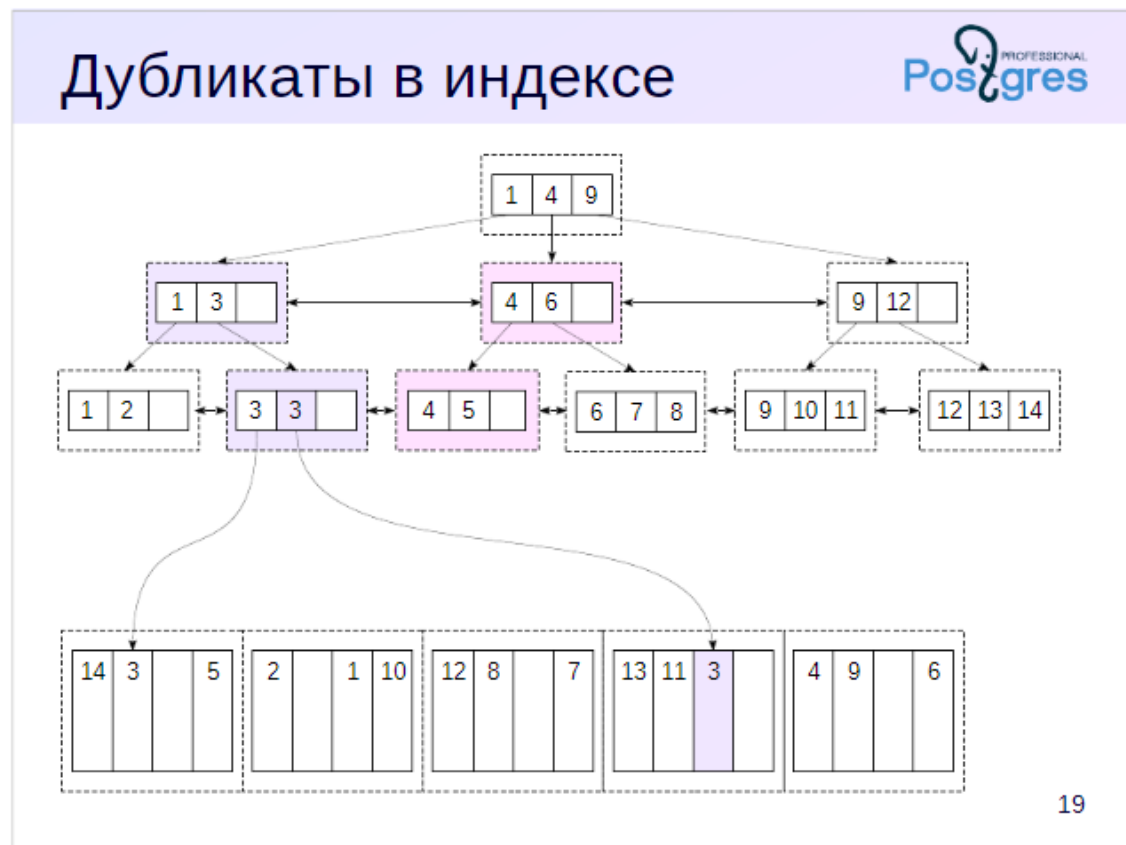
Вклад индексного доступа остается прежним.

Дальше мы не будем подробно останавливаться на расчете стоимости. Для этого используется достаточно сложная математическая модель; в нашу задачу входит только показать общий принцип.



Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html



Дубликаты в индексе — это записи с одинаковыми значениями ключей, указывающие на разные версии строк таблицы.

Дубликаты возникают по двум причинам. Во-первых, разные строки могут иметь одинаковые значения проиндексированных столбцов — в этом случае мы имеем дело с неуникальным индексом.

Во-вторых, из-за работы механизма многоверсионности могут возникать разные версии одной и той же строки — такие дубликаты постоянно возникают (и затем удаляются очисткой) и в уникальных индексах тоже.

Предположим, что в таблицу была добавлена еще одна строка со значением 3. Новую индексную запись невозможно добавить в уже заполненную листовую страницу (с ключами 3, 4, 5) и происходит расщепление: в индексе появляется еще одна листовая страница и записи распределяются между ними.

В вышестоящей внутренней странице (с ключами 1, 3, 6) должна появиться индексная запись, ссылающаяся на новую листовую страницу.

Но для этой записи нет места; поэтому внутренняя страница также расщепляется. (Для корректной работы индекса во время расщепления внутренние страницы одного уровня на самом деле тоже связаны двунаправленным списком.)

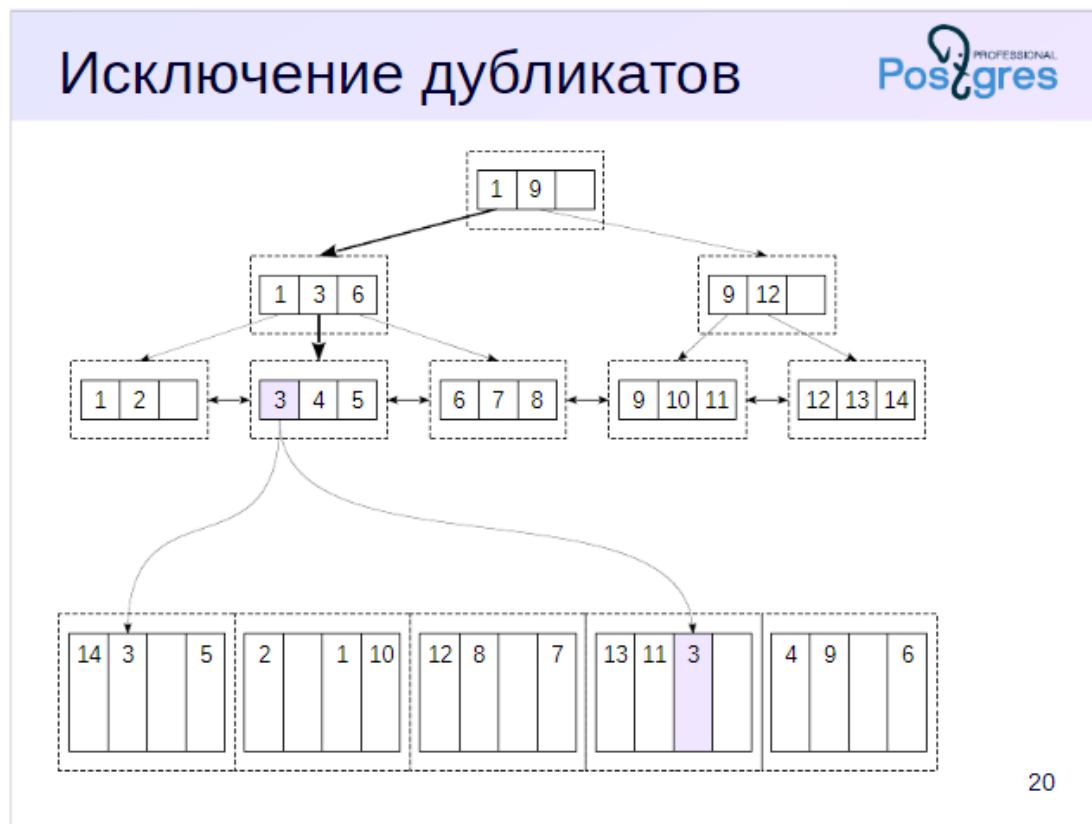
В корневой странице дерева нашлось место для новой записи, поэтому расщепления корня и увеличения глубины дерева в данном случае не произошло.

На рисунке новые страницы показаны розовым цветом.



Индексный доступ, материал из курса «PostgreSQL 13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html



Дубликаты приводят к разрастанию индекса, а расщепленные страницы никогда не объединяются.

Начиная с 13-й версии для избежания лишних расщеплений используется механизм исключения дубликатов: индексные записи с одинаковыми ключами группируются в одну общую запись со списком ссылок на версии строк.

Для экономии ресурсов группировка записей происходит только в том случае, если иначе страницу пришлось бы расщепить.

Поскольку дубликаты возникают в том числе из-за появления версий одних и тех же строк, механизм исключения дубликатов может предотвратить разрастание даже уникальных индексов. Но если известно, что дубликаты не образуются, механизм можно отключить параметром хранения `deduplicate_items`.

В нашем примере дублирующиеся тройки сгруппированы в одну индексную запись. За счет этого удалось избежать расщеплений; размер индекса не изменился.

Исключение дубликатов применяется только с типами данных, значения которых можно сравнивать побитно (не подходят типы `numeric`, `float4`, `float8`, `jsonb`; для некоторых правил сортировки не подходят текстовые типы).

Механизм не работает с Include-индексами, не реализована поддержка типов-контейнеров.

<https://postgrespro.ru/docs/postgresql/13/btree-implementation#BTREE-DEDUPLICATION>

При обновлении версии сервера с помощью `pg_upgrade` необходимо выполнить `REINDEX` для существующих индексов, чтобы задействовать для них возможность устранения дубликатов

Исключение дубликатов https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Сравним размер индекса без исключения дубликатов и с исключением. Пример подобран таким образом, что в индексе должно быть много повторяющихся значений.

Сначала создадим индекс, отключив исключение дубликатов с помощью параметра хранения `deduplicate_items`:

```
=> CREATE INDEX dedup_test ON ticket_flights (fare_conditions)
```

```
WITH (deduplicate_items = off);
```

```
CREATE INDEX
```

Посмотрим размер созданного индекса:

```
=> SELECT pg_size_pretty(pg_total_relation_size('dedup_test'));
```

```
pg_size_pretty
```

```
-----
```

```
187 MB
```

```
(1 row)
```



Исключение дубликатов https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Выполним запрос с помощью индекса, отключив для этого последовательное сканирование.

Обратите внимание на значение Buffers и время выполнения запроса.

=> SET enable_seqscan = off;

SET

=> EXPLAIN (analyze, buffers, costs off)

SELECT fare_conditions FROM ticket_flights;

QUERY PLAN

Index Only Scan using dedup_test on ticket_flights (actual time=0.034..730.144 rows=8391852 loops=1)

Heap Fetches: 0

Buffers: shared hit=4 read=23877

Planning:

Buffers: shared hit=21 read=2

Planning Time: 1.093 ms

Execution Time: 979.735 ms

(7 rows)



Исключение дубликатов https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Удалим индекс и создадим его заново. Параметр хранения deduplicate_items не указываем, так как по умолчанию он включен:

```
=> DROP INDEX dedup_test;
```

DROP INDEX

```
=> CREATE INDEX dedup_test ON ticket_flights (fare_conditions);
```

CREATE INDEX

Опять посмотрим размер индекса:

```
=> SELECT pg_size_pretty(pg_total_relation_size('dedup_test'));
```

pg_size_pretty

56 MB

(1 row)



Исключение дубликатов https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html

Размер сократился более чем в три раза.

Повторно выполним запрос:

```
=> EXPLAIN (analyze, buffers, costs off)
SELECT fare_conditions FROM ticket_flights;
QUERY PLAN
```

Index Only Scan using dedup_test on ticket_flights (actual time=0.022..490.932 rows=8391852 loops=1)

Heap Fetches: 0

Buffers: shared hit=5 read=7077

Planning:

Buffers: shared hit=5 read=1

Planning Time: 0.083 ms

Execution Time: 724.389 ms

(7 rows)

Количество Buffers тоже сократилось примерно в три раза. И запрос стал выполняться быстрее.

```
=> RESET enable_seqscan;
```

RESET



Битовые карты

Битовая карта - это вспомогательная структура данных, которая используется внутри PostgreSQL с разными целями. Битовые карты можно рассматривать как своего рода индекс: они созданы для облегчения доступа к другим структурам данных, содержащим несколько блоков. Обычно битовые карты используются для компактного представления свойств табличных данных.

На рис. показано, как битовые карты используются для доступа к данным по нескольким индексам.

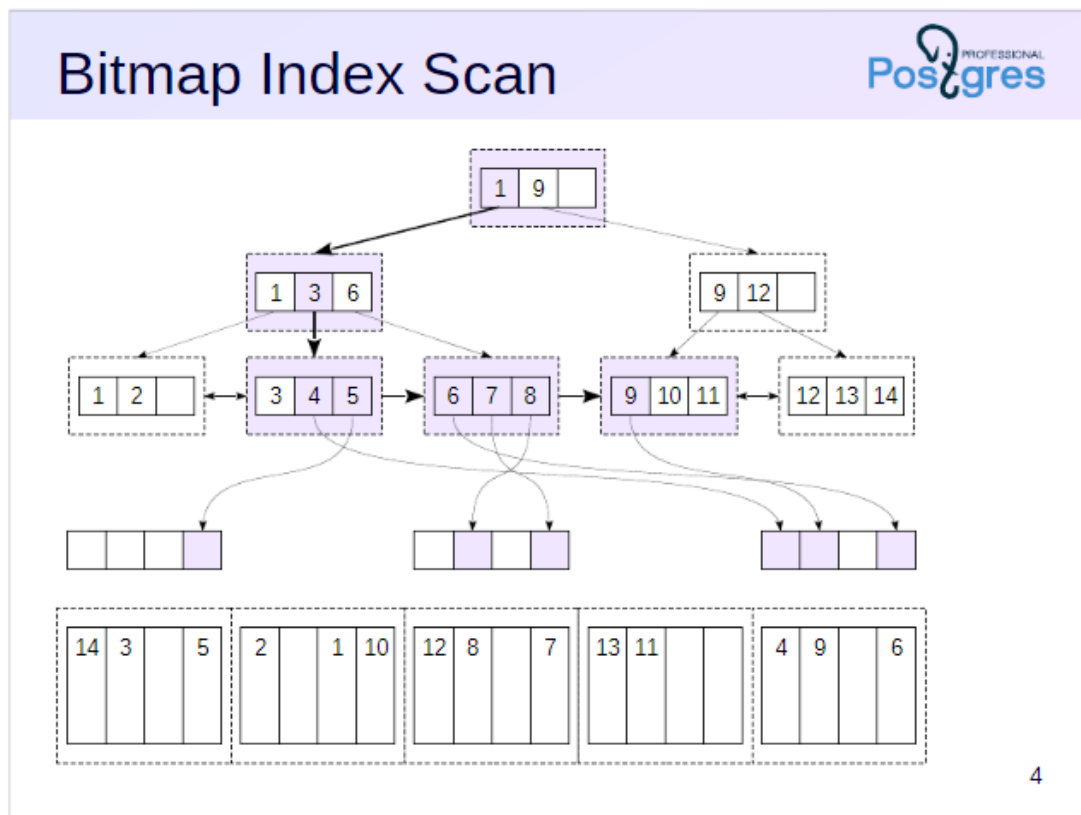
Номер блока	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Индекс 1	0	1	0	0	0	1	0	0	1	0	0	0	0	0	0	1
Индекс 2	0	0	1	0	0	1	0	0	0	1	0	0	1	0	0	1
Логическое «и» ↓																
	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1

Движок базы данных начинает со сканирования двух индексов и построения для каждого из них битовой карты, которая указывает на блоки с подходящими табличными строками. Эти битовые карты показаны на рисунке в строках, обозначенных «Индекс 1» и «Индекс 2». После их создания, движок выполняет побитовую операцию «и», чтобы найти блоки, содержащие подходящие значения для обоих критериев отбора. Затем сканируются блоки данных, соответствующие единицам в полученной битовой карте. К блокам, которые удовлетворяют только одному из двух критериев, не придется обращаться.

Битовые карты, материал из курса «PostgreSQL

13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_05_bitmapscan.html



Многочасный просмотр одних и тех же табличных страниц крайне неэффективен.

Даже в лучшем случае, если нужная страница находится в буферном кеше, ее нужно найти и заблокировать (см. курс DBA2: тема «Буферный кеш» модуля «Журналирование»), а в худшем приходится иметь дело со случайными чтениями с диска.

Чтобы не тратить ресурсы на повторный просмотр табличных страниц, применяется еще один способ доступа — сканирование по битовой карте.

Он похож на обычный индексный доступ, но происходит в два этапа.

Сначала сканируется индекс (Bitmap Index Scan) и в локальной памяти процесса строится битовая карта. Битовая карта состоит из фрагментов.

Фрагменты соответствуют табличным страницам, а каждый бит фрагмента соответствует версии строки в этой странице.

При построении битовой карты в ней отмечаются те версии строк, которые удовлетворяют условию и должны быть прочитаны.

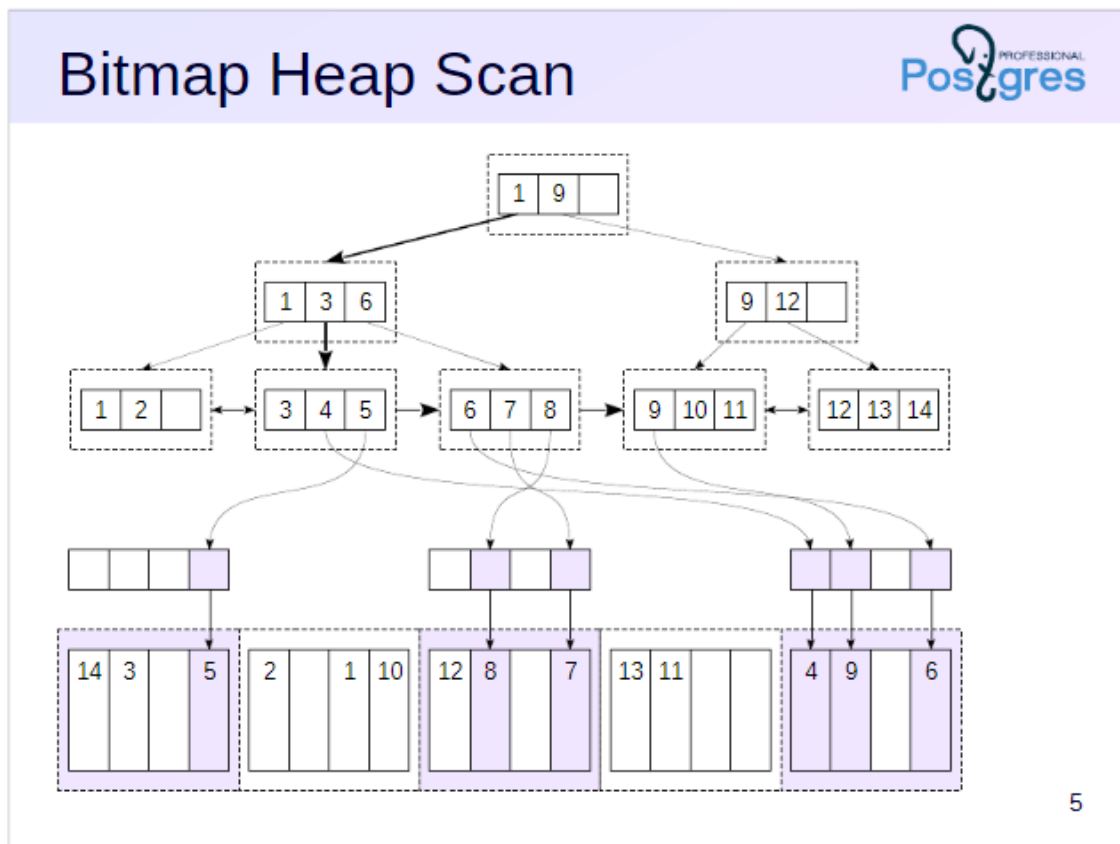
За счет разделения битовой карты на фрагменты карта, в которой отмечено немного версий, будет занимать мало места.



Битовые карты, материал из курса «PostgreSQL

13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_05_bitmapscan.html



Когда индекс просканирован и битовая карта готова, начинается сканирование таблицы (Bitmap Heap Scan).

При этом:

- используется собственный механизм предвыборки, при котором асинхронно читаются `effective_io_concurrency` страниц (по умолчанию значение равно единице);
- на одной странице может проверяться несколько версий строк, но каждая страница просматривается ровно один раз.



Битовые карты https://edu.postgrespro.ru/qpt-13/qpt_05_bitmapscan.html

Сканирование по битовой карте

Будем рассматривать таблицу бронирований bookings.

Создадим на ней два дополнительных индекса:

```
=> CREATE INDEX ON bookings(book_date);
```

```
CREATE INDEX
```

```
=> CREATE INDEX ON bookings(total_amount);
```

```
CREATE INDEX
```

Посмотрим, какой метод доступа будет выбран для поиска диапазона.

```
=> EXPLAIN SELECT * FROM bookings WHERE total_amount < 10000;
```

QUERY PLAN

Bitmap Heap Scan on bookings (cost=1145.71..15356.08 rows=61069 width=21)

Recheck Cond: (total_amount < '10000'::numeric)

-> Bitmap Index Scan on bookings_total_amount_idx (cost=0.00..1130.45 rows=61069 width=0)

Index Cond: (total_amount < '10000'::numeric)

(4 rows)

*Выбран метод доступа Bitmap Scan.
Он состоит из двух узлов:*

- Bitmap Index Scan читает индекс и строит битовую карту;*
- Bitmap Heap Scan читает табличные страницы, используя построенную карту.*

Обратите внимание, что карта должна быть построена полностью, прежде чем ее можно будет использовать.



Битовые карты https://edu.postgrespro.ru/qpt-13/qpt_05_bitmapscan.html

Объединение битовых карт

Кроме того, что битовая карта позволяет избежать повторных чтений табличных страниц, с ее помощью можно объединять нескольких условий.

=> EXPLAIN (costs off)

```
SELECT * FROM bookings
```

```
WHERE total_amount < 10000 OR total_amount > 100000;
```

QUERY PLAN

Bitmap Heap Scan on bookings

Recheck Cond: ((total_amount < '10000'::numeric) OR (total_amount > '100000'::numeric))

-> BitmapOr

-> Bitmap Index Scan on bookings_total_amount_idx

Index Cond: (total_amount < '10000'::numeric)

-> Bitmap Index Scan on bookings_total_amount_idx

Index Cond: (total_amount > '100000'::numeric)

(7 rows)

Здесь сначала были построены две битовые карты — по одной на каждое условие, а затем объединены побитовой операцией «или».



Битовые карты https://edu.postgrespro.ru/qpt-13/qpt_05_bitmapscan.html

Таким же образом могут быть использованы и разные индексы.

```
=> EXPLAIN (costs off)
SELECT * FROM bookings
WHERE total_amount < 10000
      OR book_date = bookings.now() - INTERVAL '1 day';
```

QUERY PLAN

Bitmap Heap Scan on bookings

Recheck Cond: ((total_amount < '10000'::numeric) OR (book_date = ('2017-08-15 18:00:00+03'::timestamp with time zone - '1 day'::interval)))

-> BitmapOr

-> Bitmap Index Scan on bookings_total_amount_idx

Index Cond: (total_amount < '10000'::numeric)

-> Bitmap Index Scan on bookings_book_date_idx

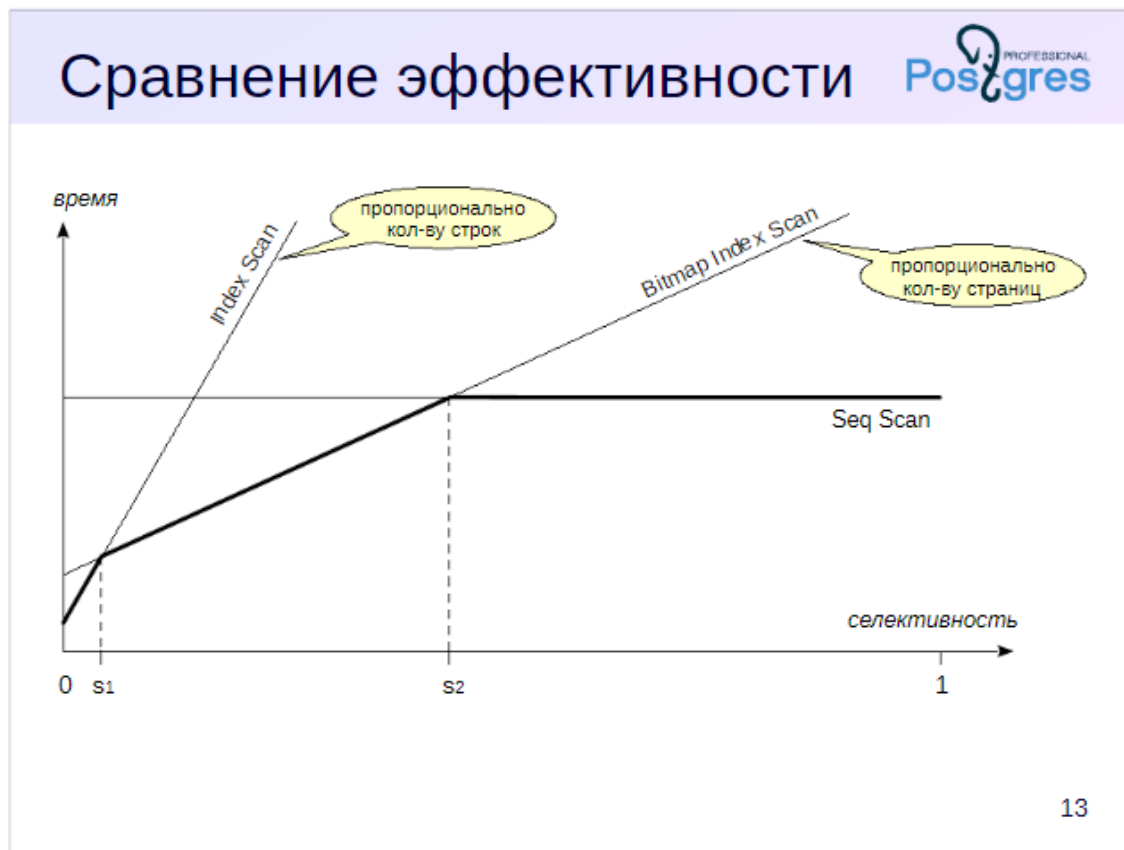
Index Cond: (book_date = ('2017-08-15 18:00:00+03'::timestamp with time zone - '1 day'::interval))
(7 rows)



Битовые карты, материал из курса «PostgreSQL

13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_05_bitmapscan.html



Индексное сканирование лучше всего работает при очень высокой селективности, когда по индексу выбирается одно или несколько значений.

При средней селективности лучше всего показывает себя сканирование по битовой карте.

Оно работает лучше индексного сканирования, поскольку обходится без повторных чтений одних и тех же страниц.

Однако при сканировании по битовой карте возникают накладные расходы на построение карты, поэтому при высокой селективности оно проигрывает.

При низкой селективности лучше всего работает последовательное сканирование: если надо выбрать все или почти все табличные строки, обращение к индексным страницам только увеличивает накладные расходы. Этот эффект усиливается в случае вращающихся дисков, где стоимость произвольного чтения существенно выше стоимости чтения последовательно расположенных страниц.

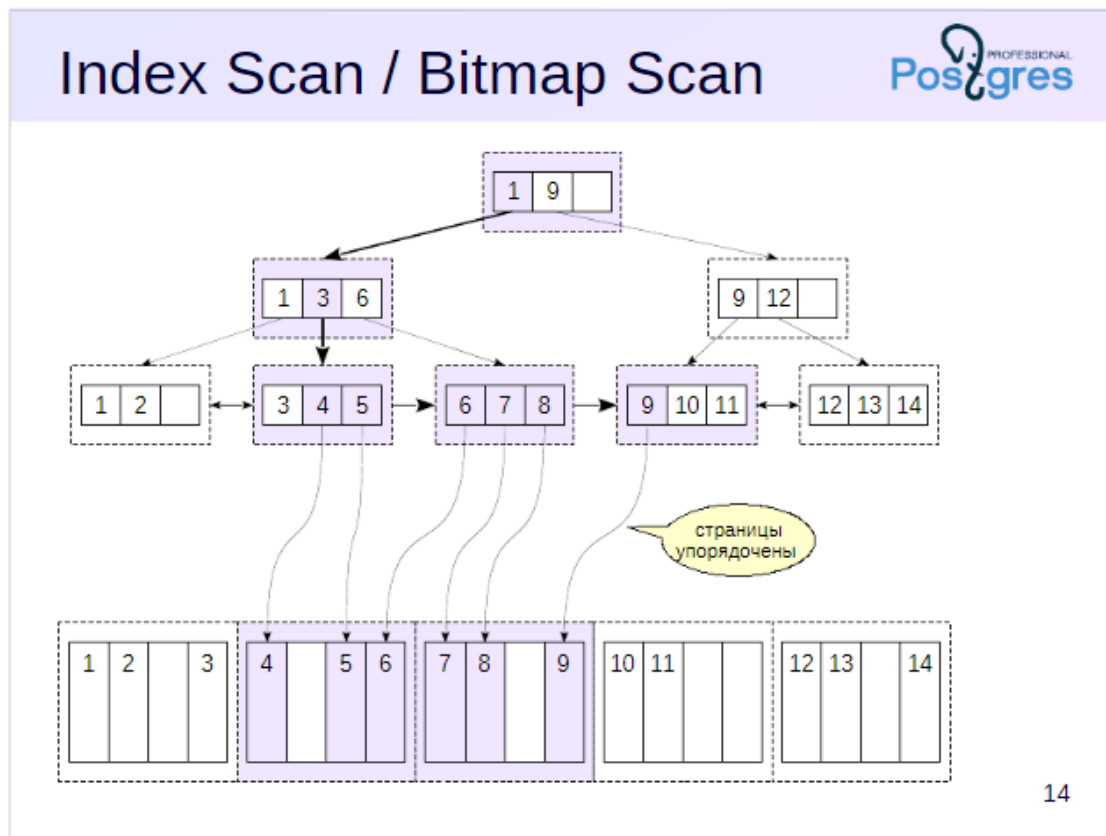
Значение селективности, при котором становится выгодно переключиться на другой метод доступа, сильно зависит от конкретной таблицы и конкретного индекса. Планировщик учитывает множество параметров, чтобы выбрать наиболее подходящий способ.

Еще одно замечание: при индексном доступе результат возвращается отсортированным, что в ряде случаев увеличивает привлекательность такого доступа даже при низкой селективности.

Битовые карты, материал из курса «PostgreSQL

13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_05_bitmapscan.html



Если данные в таблице физически упорядочены, обычное индексное сканирование не будет читать табличную страницу повторно. В таком (не частом на практике) случае метод сканирования по битовой карте теряет смысл, проигрывая обычному индексному сканированию.

Разумеется, планировщик это тоже учитывает (как именно - рассматривается в теме «Статистика»).



Кластеризация https://edu.postgrespro.ru/qpt-13/qpt_05_bitmapscan.html

Если строки таблицы упорядочены так же, как и индекс, битовая карта становится излишней. Продемонстрируем это с помощью команды CLUSTER.

Сейчас строки таблицы физически упорядочены по номеру бронирования:

=> `SELECT * FROM bookings LIMIT 10;`

book_ref	book_date	total_amount
-----+-----+-----		
000004	2016-08-13 15:40:00+03	55800.00
00000F	2017-07-05 03:12:00+03	265700.00
000010	2017-01-08 19:45:00+03	50900.00
000012	2017-07-14 09:02:00+03	37900.00
000026	2016-08-30 11:08:00+03	95600.00
00002D	2017-05-20 18:45:00+03	114700.00
000034	2016-08-08 05:46:00+03	49100.00
00003F	2016-12-12 15:02:00+03	109800.00
000048	2016-09-17 01:57:00+03	92400.00
00004A	2016-10-13 21:57:00+03	29000.00

(10 rows)

Переупорядочим строки в соответствии с индексом по столбцу total_amount.



Кластеризация https://edu.postgrespro.ru/qpt-13/qpt_05_bitmapscan.html

Пока идет процесс, обратите внимание:

Команда CLUSTER устанавливает исключительную блокировку, поскольку полностью перестраивает таблицу (как VACUUM FULL);

Строки упорядочиваются, но не поддерживаются в упорядоченном виде — в процессе работы кластеризация будет ухудшаться.

```
=> CLUSTER bookings USING bookings_total_amount_idx;
```

CLUSTER

```
=> ANALYZE bookings;
```

ANALYZE



Кластеризация https://edu.postgrespro.ru/qpt-13/qpt_05_bitmapscan.html

Убедимся, что строки упорядочены по стоимости:

⇒ `SELECT * FROM bookings LIMIT 10;`

book_ref	book_date	total_amount
00F39E	2017-02-12 04:11:00+03	3400.00
0103E1	2017-04-03 09:32:00+03	3400.00
013695	2016-11-01 09:30:00+03	3400.00
0158C0	2017-04-24 07:47:00+03	3400.00
01AD57	2017-03-15 16:11:00+03	3400.00
020E97	2017-01-02 12:25:00+03	3400.00
021FD3	2017-05-27 10:20:00+03	3400.00
0278C7	2017-02-04 17:42:00+03	3400.00
029452	2016-12-10 13:51:00+03	3400.00
0355DD	2016-09-19 13:22:00+03	3400.00

(10 rows)



Кластеризация https://edu.postgrespro.ru/qpt-13/qpt_05_bitmapscan.html

=> EXPLAIN SELECT * FROM bookings WHERE total_amount < 10000;

QUERY PLAN

Index Scan using bookings_total_amount_idx on bookings (cost=0.43..2215.86 rows=63339 width=21)

Index Cond: (total_amount < '10000'::numeric)

(2 rows)

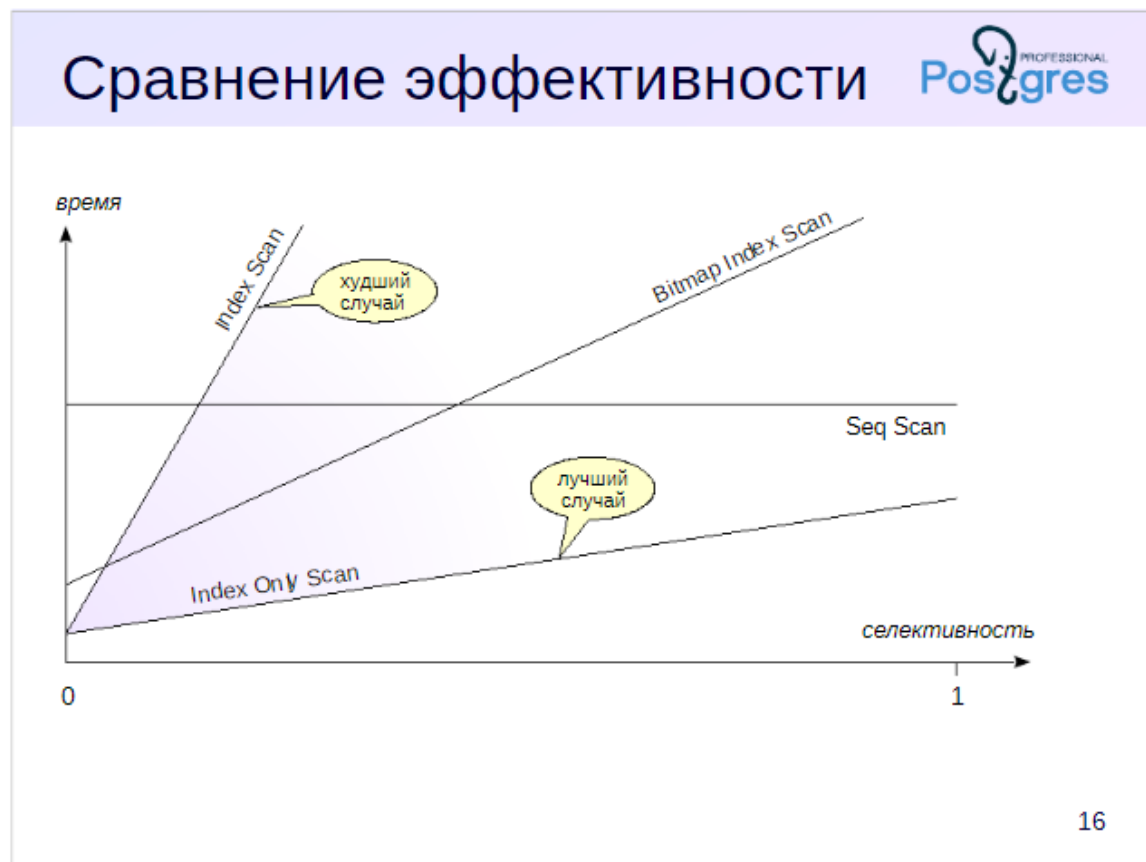
До кластеризации использовалось сканирование по битовой карте, но теперь проще и выгодней сделать обычное индексное сканирование.



Битовые карты, материал из курса «PostgreSQL

13. Оптимизация запросов»:

https://edu.postgrespro.ru/qpt-13/qpt_05_bitmapscan.html



Эффективность сканирования только индекса сильно зависит от актуальности карты видимости (и от того, сколько страниц действительно содержат только актуальные версии строк).

В лучшем случае этот метод доступа может быть эффективней последовательного сканирования даже при низкой селективности (если индекс меньше, чем таблица, и особенно на SSD-дисках).

В худшем случае, когда требуется проверять видимость каждой строки, метод вырождается в обычное индексное сканирование.

Поэтому планировщику приходится учитывать состояние карты видимости: при неблагоприятном прогнозе исключительно индексное сканирование не применяется из-за опасности получить замедление вместо ускорения.



Другие виды индексов

PostgreSQL предлагает множество индексных структур, поддерживающих разные типы данных и разные классы условий поиска.

Хеш-индекс использует хеш-функцию для вычисления адреса индексного блока, содержащего индексный ключ. Для условия равенства этот тип индекса работает быстрее В-дерева, однако он совершенно бесполезен для запросов по диапазону. Стоимостная оценка поиска по хеш-индексу не зависит от размера индекса (в отличие от логарифмической зависимости для В-деревьев).

R-дерево поддерживает поиск по пространственным данным. Индексный ключ для R-дерева всегда представляет собой прямоугольник в многомерном пространстве. Поиск возвращает все объекты, имеющие непустое пересечение с прямоугольником запроса. Структура R-дерева похожа на структуру В-дерева; однако расщепление переполненных узлов устроено намного сложнее. R-деревья эффективны для небольшого числа измерений (обычно от двух до трех).

Другие типы индексов, доступные в PostgreSQL, полезны для полнотекстового поиска, поиска в очень больших таблицах и многого другого.



Другие виды индексов

PostgreSQL предлагает множество индексных структур, поддерживающих разные типы данных и разные классы условий поиска.

Хеш-индекс использует хеш-функцию для вычисления адреса индексного блока, содержащего индексный ключ. Для условия равенства этот тип индекса работает быстрее В-дерева, однако он совершенно бесполезен для запросов по диапазону. Стоимостная оценка поиска по хеш-индексу не зависит от размера индекса (в отличие от логарифмической зависимости для В-деревьев).

R-дерево поддерживает поиск по пространственным данным. Индексный ключ для R-дерева всегда представляет собой прямоугольник в многомерном пространстве. Поиск возвращает все объекты, имеющие непустое пересечение с прямоугольником запроса. Структура R-дерева похожа на структуру В-дерева; однако расщепление переполненных узлов устроено намного сложнее. R-деревья эффективны для небольшого числа измерений (обычно от двух до трех).

Другие типы индексов, доступные в PostgreSQL, полезны для полнотекстового поиска, поиска в очень больших таблицах и многого другого.



Задание (см. также https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html), вариант 1, вариант выбираете по формуле: номер_по_журналу % 2 + 1

1. Проверьте, выполняется ли параллельно запрос, вычисляющий суммарную стоимость билетов, каждый из которых дешевле 5000. Поясните элементы плана запроса.
2. Проверьте, как выполняется (параллельно или последовательно, почему) запрос из п. 1, если чтение из таблицы перелетов оформлено как общее табличное выражение (CTE). Используйте указание MATERIALIZED (WITH CTEname AS MATERIALIZED (SELECT ... См. 26 слайд), чтобы планировщик не раскрывал общее табличное выражение. Поясните элементы плана запроса.
3. Напишите двумя способами (с помощью агрегатной функции, с помощью сортировки и LIMIT) запрос, выбирающий минимальную цену билета. Проверьте план выполнения. Какой метод доступа выбрал Планировщик? Эффективен ли такой доступ? Создайте индекс по столбцу ticket_flights.amount. Снова проверьте план выполнения запроса. Какой метод доступа выбрал планировщик теперь? Объясните план запроса.
4. Создайте include-индекс для таблицы flights (по полю flight_id с любым неключевым столбцом). Проанализируйте план запроса, использующего данные из ключевого столбца и выбранного неключевого, объясните его. Замените созданным индексом индекс, поддерживающий первичный ключ таблицы, не забудьте сохранить целостность данных.
5. Напишите запрос, находящий информацию о перелетах стоимостью более 150 000 руб. Какой метод доступа был выбран? Сколько времени выполняется запрос? Запретите выбранный метод доступа, снова выполните запрос и сравните время выполнения. Для запрета метода доступа установите один из параметров в значение off: - enable_seqscan, - enable_indexscan, - enable_bitmapscan. Например: SET enable_seqscan = off; Прав ли был оптимизатор?
6. Выполните задания 3-4, 6-7, 14 со стр. 317-325 учебника Моргунов, Е. П. PostgreSQL. Основы языка SQL: учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова, П. В. Лузанова. — СПб.: БХВ-Петербург, 2018. — 336 с.: ил. (<https://postgrespro.ru/education/books/sqlprimer>)



Задание (см. также https://edu.postgrespro.ru/qpt-13/qpt_04_indexscan.html), вариант 2, вариант выбираете по формуле: номер_по_журналу % 2 + 1

1. Проверьте, выполняется ли параллельно запрос, вычисляющий суммарную стоимость билетов, каждый из которых дороже 7000. Поясните элементы плана запроса.
2. Проверьте, как выполняется (параллельно или последовательно, почему) запрос из п. 1, если чтение из таблицы перелетов оформлено как общее табличное выражение (CTE). Используйте указание MATERIALIZED (WITH CTEname AS MATERIALIZED (SELECT ... См. 26 слайд), чтобы планировщик не раскрывал общее табличное выражение. Поясните элементы плана запроса.
3. Напишите двумя способами (с помощью агрегатной функции, с помощью сортировки и LIMIT) запрос, выбирающий максимальную цену билета. Проверьте план выполнения. Какой метод доступа выбрал Планировщик? Эффективен ли такой доступ? Создайте индекс по столбцу ticket_flights.amount. Снова проверьте план выполнения запроса. Какой метод доступа выбрал планировщик теперь? Объясните план запроса.
4. Создайте include-индекс для таблицы flights (по полю flight_id с любым неключевым столбцом). Проанализируйте план запроса, использующего данные из ключевого столбца и выбранного неключевого, объясните его. Замените созданным индексом индекс, поддерживающий первичный ключ таблицы, не забудьте сохранить целостность данных.
5. Напишите запрос, находящий информацию о перелетах стоимостью более 200 000 руб. Какой метод доступа был выбран? Сколько времени выполняется запрос? Запретите выбранный метод доступа, снова выполните запрос и сравните время выполнения. Для запрета метода доступа установите один из параметров в значение off: - enable_seqscan, - enable_indexscan, - enable_bitmapscan. Например: SET enable_seqscan = off; Прав ли был оптимизатор?
6. Выполните задания 3-4, 6-7, 14 со стр. 317-325 учебника Моргунов, Е. П. PostgreSQL. Основы языка SQL: учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова, П. В. Лузанова. — СПб.: БХВ-Петербург, 2018. — 336 с.: ил. (<https://postgrespro.ru/education/books/sqlprimer>)

